Санкт-Петербургский государственный университет

ГРИДЯЕВА Полина Александровна

Выпускная квалификационная работа

Методы машинного обучения в исследовании энергетических характеристик единичных событий в коллайдерном эксперименте

Уровень образования: бакалавриат
Направление 03.03.01 «Прикладные математика и физика»
Основная образовательная программа СВ.5009.2021 «Прикладные физика и математика»

Научный руководитель:
профессор, кафедра
вычислительной физики,
д.ф.-м.н., Руднев Владимир
Александрович

Рецензент: профессор, кафедра
ядерно-физических методов
исследования, д.ф.-м.н.,
Валиев Фархат Фагимович

Санкт-Петербург
2025

Оглавление

Введение ... 3
1. Искусственные нейронные сети ... 5
2. Модель эксперимента ... 7
3. Реализация ... 8
4. Проверка выполнения ЗСЭ и ЗСИ ... 13
5. Фильтры энергии и импульса ... 15
6. Построение и анализ зависимости энергии взаимодействия от прицельного параметра ... 16
7. Построение функции, отображающей сигнал с детектора на выделившуюся энергию ... 21
Выводы и дальнейшие направления исследования ... 29
Используемое оборудование ... 31
Литература ... 32
Приложение 1 ... 33
Приложение 2 ... 34

Введение

Физика высоких энергий – это область науки, изучающая фундаментальные взаимодействия элементарных частиц и атомных ядер при энергиях, значительно превышающих их энергии покоя [1]. Данные исследования позволяют не только заглянуть в природу материи на микроскопическом уровне, но и воспроизвести условия, существовавшие во Вселенной в первые мгновения после Большого взрыва. Современные теоретические модели, такие как Стандартная модель и ее расширения, требуют экспериментального подтверждения, что делает физику высоких энергий одним из ключевых направлений фундаментальной науки. Экспериментальная физика в этой области основана на использовании ускорителей частиц, разгоняющих заряженные частицы до скоростей, близких к скорости света, и сталкивающих их при определенных условиях. Важнейшими объектами для исследований являются протоны, тяжелые ионы, а также антипротоны и позитроны, взаимодействия которых позволяют изучать кварк-глюонную плазму, механизмы нарушения симметрий и свойства новых частиц.
При анализе результатов столкновений особое внимание уделяется процессу регистрации и интерпретации данных. Не все параметры взаимодействия могут быть измерены напрямую. Например, такие важные характеристики, как прицельный параметр (impact parameter), который определяет степень центральности столкновения, или координата взаимодействия вдоль оси пучка, вычисляются косвенными методами. Их точное определение играет ключевую роль в исследовании многочастичных взаимодействий и верификации теоретических моделей [2].
Кроме того, обработка данных в физике высоких энергий требует применения современных методов анализа больших объемов информации. Для этого используются алгоритмы машинного обучения и статистические методы, позволяющие эффективно фильтровать экспериментальные события, выделять редкие процессы и моделировать динамику взаимодействий частиц. Оптимизация этих методов способствует более точному восстановлению параметров столкновений и поиску новых физических явлений. Применение различных методов анализа данных имеет критическое значение в расширении наших представлений о фундаментальных законах природы.

В модели столкновения, рассматриваемой в данной работе, в коллайдере взаимодействуют ядра тяжелых ионов и в ходе этого процесса образуется большое количество вторичных частиц, распространяющихся в разные стороны от зоны взаимодействия. Используя параметры, регистрируемые микроканальными пластинчатыми детекторами (например, время пролета, координаты столкновения частицы с детектором, номер детектора), можно построить физическую модель события, оценив основные характеристики столкновения, в том числе выделившуюся в ходе столкновения энергию.

Одним из эффективных подходов к решению задач регрессии в области анализа ядерных столкновений является применение глубоких нейронных сетей. В частности, в работе [3] была предложена модель глубокой полносвязной нейронной сети, способной предсказывать коэффициент эллиптического потока на основе параметров события, таких как центральность и поперечный импульс частиц. В другой работе [4] авторы применили сверточную нейронную сеть для восстановления физических величин, включая энергию, по пространственному распределению плотности энергии в ультрарелятивистских столкновениях тяжёлых ионов. Оба подхода демонстрируют высокую точность и обосновывают актуальность использования нейросетевых методов в задачах предсказания характеристик столкновений.

Целью настоящей работы является оценка на событийном уровне энергии частиц, провзаимодействовавших в ходе столкновения двух ядер золота, с использованием

технологий машинного обучения, на основе данных, получаемых с детекторов на микроканальных пластинах, чтобы в дальнейшем поставить эти детекторы на MPD NICA [4]. В качестве модели динамики столкновений используется генератор QGSM.

Для достижения данной цели были поставлены следующие задачи:

1. Проверить выполнение законов сохранения энергии и импульса для генератора QGSM при лобовом столкновении ядер золота (Au-Au) с начальной энергией $\sqrt{s_{NN}} = 11.006\ \mathrm{GeV}$ на нуклон-нуклонную пару.
2. Учитывая оценку ЗС, сформулировать критерии, по которым необходимо определять энергию, выделившуюся в ходе столкновения.
3. Построить графики зависимости выделившейся энергии в ходе взаимодействия в зависимости от прицельного параметра столкновения и объяснить их физический смысл.
4. Построить функцию, способную отображать сигнал с микроканальных пластинчатых детекторов на выделившуюся в столкновении энергию.

1. Искусственные нейронные сети

Искусственная нейронная сеть (ИНС) – это математическая модель, вдохновленная работой реальной биологической нейронной сети: мозг получает информацию из внешнего мира с помощью всех органов чувств, а затем эта информация передается между связанными друг с другом узлами - нейронами.
Нейросеть это функция, которая преобразовывает входные данные, обрабатывая их с помощью одного или нескольких скрытых слоев, и передает преобразованные данные на выход. Каждый нейрон в ИНС выполняет ряд простых преобразований: взвешенное суммирование входов, добавление смещения, применение функции активации. Выходные данные с одного слоя ИНС можно описать следующей формулой:

$$\boldsymbol{y} = \sigma(\boldsymbol{W} \cdot \boldsymbol{x} + \boldsymbol{b}),$$

Где $\boldsymbol{x} \in \mathbb{R}^n -$ входной вектор, $\boldsymbol{W} \in \mathbb{R}^{m \times n}$ – матрица весов, $\boldsymbol{b} \in \mathbb{R}^n$ – вектор смещений, $\sigma(\cdot) -$ функция активации.

В данной работе рассматривались два типа нейронных сетей: полносвязные и гибридные (с добавлением сверточного слоя). Сеть считается полносвязной, если в слое каждый нейрон связан со всеми нейронами из предыдущего слоя. Полносвязные нейронные сети являются универсальный аппроксимантом, способным приближать произвольные непрерывные функции с заданной точностью. Это делает их эффективным инструментом для решения задач регрессии и, как следствие, их использование в данной работе оправдано. В случае сверточной нейронной сети вместо полной связности применяется свертка небольших обучаемых фильтров ко входным данным. Формально выход сверточного слоя задается следующей формулой:

$$out(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{l-1} input(i+m, j+n) \cdot kernel(m,n), \text{ где}$$

$k \times l -$ размер ядра свертки, $input -$ входные признаки, $out -$ выходные данные, $kernel -$ это фильтр, который применяется к входным данным.

Сверточные нейронные сети (CNN) зарекомендовали себя как эффективный инструмент в задачах компьютерного зрения и широко применяются при анализе изображений. Данные, поступающие с детекторов, могут быть представлены в форме двумерного массива, аналогично изображениям, поэтому использование сверточных слоев также является логичным и обоснованным.

В качестве функции активации использовалась функция SiLU (Sigmoid Linear Unit):

$$SiLU(x) = x \cdot \sigma(x) = \frac{x}{1+e^{-x}},$$

где $\sigma(x) -$ это сигмоидная функция активации.

В отличие от других функций активации, таких как ReLU, SiLU является гладкой и дифференцируемой на всей числовой прямой, что способствует повышению качества обучения модели.
Функция активации нужна для того, чтобы добавить нелинейность в модель. Это помогает извлекать сложные зависимости в исследуемых данных.

В ходе обучения модели происходит оптимизация параметров обучения: нейронов в слоях, весов и смещений. Цель обучения – минимизировать функцию ошибки, которая отражает степень расхождения между предсказанными моделью значениями и истинными значениями. В данной работе в качестве функции потерь используется среднеквадратичная ошибка (Mean Squared Error, MSE):

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\widehat{y_i} - y_i)^2, \text{где}$$

$\widehat{y_i}$ – это предсказанное значение, а y_i – это истинное значение, N – общее количество наблюдений (примеров в наборе данных для обучения НС).

Благодаря функции потерь можно оценить то, насколько хороший или плохой результат выдает рассматриваемая модель. Минимизация этой функции позволяет повысить точность модели и обеспечить ее способность находить и обобщать закономерности, выявленные на обучающем наборе данных.

Для оптимизации параметров модели в данной работе используется оптимизатор $Adam$ (Adaptive Momentum Estimation). $Adam$ автоматически подстраивает шаг градиентного спуска на каждом этапе обучения, что повышает устойчивость модели, а также снижает вероятность “застревания” в локальных минимумах. Именно благодаря оптимизатору удается достичь более стабильных и воспроизводимых результатов при обучении нейронной сети.

Обучение же модели в ML бывает нескольких типов. Самые распространенные – это обучение с учителем, без учителя и с подкреплением. В данной работе используется обучение с учителем (supervised learning) – тип машинного обучения, при котором модель обучается на размеченных данных, то есть на парах входных признаков и соответствующих целевых значений. В дальнейшем модель пытается совершать предсказания для новых, ранее не встречавшихся данных, основываясь на приобретенной на обучающей выборке способности обобщать зависимости и искать закономерности. В рамках данной работы используется обучение с учителем, поскольку одной из задач является предсказание числовых значений – выделившейся в ходе ядерного столкновения энергии – по известным характеристикам событий.

Как уже было затронуто ранее, для оценки качества работы модели и предотвращения переобучения исходный набор данных может быть разделен на тренировочную и тестовую выборки. В данной работе деление происходит в соотношении 80/20. То есть 80% данных используются для обучения модели – оптимизации ее внутренних параметров (весов, смещений), а 20% - для проверки того, насколько хорошо модель справляется с задачей на новых данных, никогда ранее не виденных ею. Разделение выборки необходимо для того, чтобы оценить обобщающую способность модели, а не только ее способность запоминать обучающие данные. Если же модель плохо справляется с поставленной задачей на тестовой выборке, то можно говорить о переобучении (overfitting) – ситуации, когда модель слишком точно подстроилась под обучающую выборку и потеряла способность к обобщению.

В контексте данной работы модель учится предсказывать выделившуюся в ходе столкновения двух ядер золота энергию, используя характеристики событий. Это так называемая задача регрессии – тип задачи в машинном обучении, в которой модель обучается предсказывать непрерывные значения на основе входных данных.

2. Модель эксперимента

В данном эксперименте рассматриваются столкновения двух ядер золота Au+Au ($A = 197$ – атомная масса, $Z = 79$ – заряд ядра золота) при высоких энергиях. В момент столкновения происходит высвобождение энергии, сопровождаемое образованием вторичных частиц, которые в дальнейшем разлетаются во все стороны и фиксируются детекторной системой. Столкновения в данной работе являются одиночными, а также рассматриваются в системе центра масс. Для каждого события детектор регистрирует время пролета (time-of-flight) и координаты столкновения частицы с поверхностью детектора. Зарегистрированная информация позволяет судить о пространственно-временной структуре события, а также по совокупности полученных сигналов можно восстановить физические характеристики самого столкновения – в частности, энергию, выделившуюся в результате столкновения. Такой подход лежит в основе многих задач анализа данных в физике высоких энергий: выделившаяся энергия не наблюдается напрямую, а определяется исходя из информации, получаемой с детекторов.

Для подготовки данных эксперимента, а также для дальнейшего анализа энергетических характеристик единичных событий в коллайдерном эксперименте очень важна конфигурация детекторной системы. В данной работе детекторы на микроканальных пластинах расположены на одинаковом расстоянии от точки столкновения ядер золота равном $z_1 = z_2 = z = 400$ см. Помимо этого детекторы помещены внутрь ускорителя и имеют кольцевую форму: диаметр внешнего кольца – $D = 50$ см, диаметр внутреннего кольца – $d = 5$ см. Кроме того, детекторы разбиты на чувствительные пластины (пады) по углу и по радиусу. Каждый пад способен зарегистрировать информацию о частице, попавшей на него, независимо от других чувствительных пластин. Стоит отметить, что каждый пад способен регистрировать только частицу, которая первой долетела до него. Мы разбиваем детекторы на 60 падов: берем 6 колец и 10 секторов.

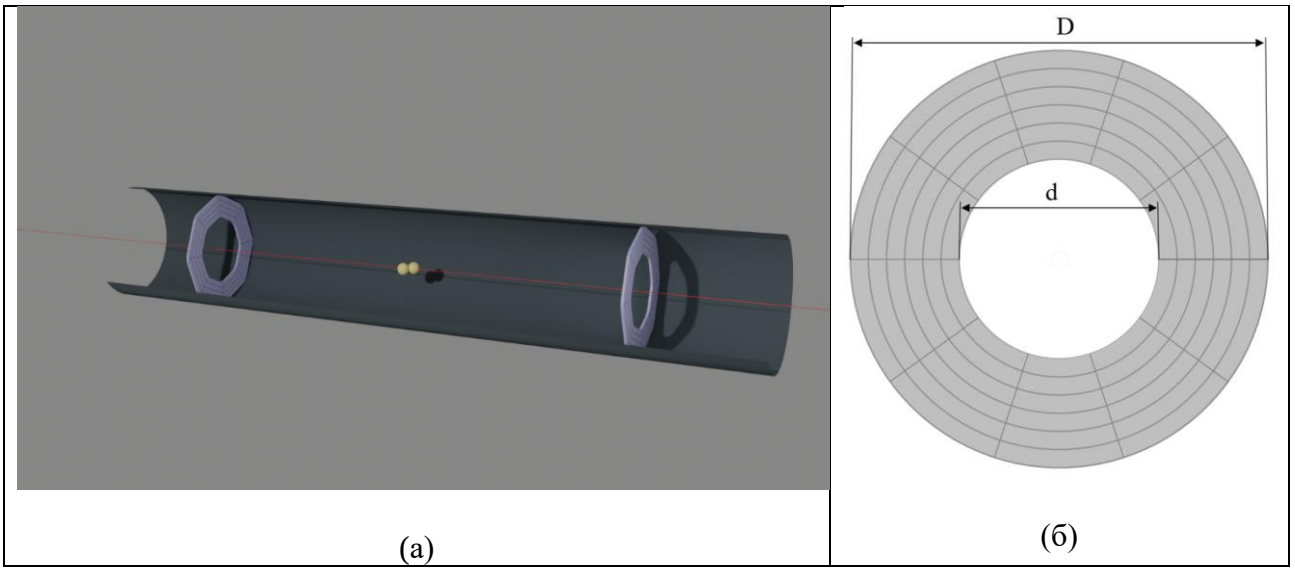


Рис. 1: Схема компонентов детекторной установки: а – схема расположения детекторов относительно точки столкновения ядер, б – разбиение на чувствительные пады по углу и радиусу.

Приведем теперь часть активно используемых в ходе работы формул и постоянных. Для начального состояния справедливы следующие значения энергий:

$\sqrt{s_0} = 5.5034\ GeV$ – энергия на один нуклон

$\sqrt{s_{NN}} = 11.0067\ GeV$ – энергия на нуклон-нуклонную пару

$\sqrt{s_{total}} = 2 \cdot A \cdot \sqrt{s_0} = 2 \cdot 197 \cdot 5.5034 = 2168\ GeV$ – энергия двух ядер золота

Одной из задач этой работы является исследование зависимости выделившейся в ходе столкновения энергии от прицельного параметра. Прицельный параметр (b) в данной модели – это расстояние между центрами двух ядер золота. Рассмотрим используемые для обработки данных формулы:

Суммарная энергия частиц, образовавшихся в ходе столкновения [Приложение 1]:

$$\sqrt{s_{eff}}(b) = \frac{N_{part}}{2A}\sqrt{s_{total}} = N_{part} * \sqrt{s_0} \quad (1)$$

Энергия каждой частицы в отдельности

$$energy_{csm} = \sqrt{p_x^2 + p_y^2 + p_z^2 + m^2} \quad (2)$$

Ожидаемая зависимость $\sqrt{s_{eff}}(b)$:

$$\sqrt{s_{eff}} = C \cdot (R^2 - b^2)^2 \quad (3)$$

где C – константа, R – радиус золота, b – прицельный параметр [Приложение 1].

3. Реализация

Рассмотрев устройство детекторной системы, а также физику процесса (распространение частиц от точки столкновения до плоскости детектора), перейдем к формированию вектора признаков, подаваемого на вход нейронной сети. Вектор признаков содержит числовое представление о событие столкновения, включая в себя важные параметры, которые можно извлечь из сигнала, получаемого с микроканальных пластинчатых детекторов. Эти параметры играют ключевую роль в процессе реконструкции коллайдерных событий, а также в анализе интересующих исследователей величин. В контексте данной работы основное внимание уделяется выделившейся в ходе столкновения энергии.

Вектор признаков формируется на основе трех основных величин, измеряемых для каждой зафиксированной детектором частицы:

1. Время пролета (time-of-flight, tof). Зная энергию и импульс частицы можно вычислить ее скорость и время пролета и, как следствие, сделать вывод о ее природе – например, отличить легкие частицы (электроны, мюоны) от более тяжелых (протоны, пионы).
2. Номер сработавшего пада (pad number). Данный параметр показывает куда именно на поверхности детектора попала частица.
3. Номер детектора (detector ID). В нашей конфигурации детекторов два: слева и справа от точки столкновения. В какой именно детектор попала частица будем определять по направлению проекции импульса p_z: если $p_z > 0$, то detector ID = 1, если $p_z < 0$, то detector ID = -1.

Для восстановления координат столкновения частицы с детектором считаем, что траекторию движения частиц можно линейно аппроксимировать. Введем параметр движения частицы t, связанный с импульсом $\vec{p} = (p_x, p_y, p_z)$:

$$\begin{cases} x = t \cdot p_x \\ y = t \cdot p_y \\ z = t \cdot p_z \end{cases},$$

Поскольку детектор расположен в плоскости $z_0 = |z| = 400$ см, параметр движения задается следующий выражением: $t = \frac{z_0}{p_z}$. Зная этот параметр, можно определить координаты точки столкновения частицы с детектором:

$$x = \frac{z_0}{p_z} \cdot p_x, \qquad y = \frac{z_0}{p_z} \cdot p_y$$

Очевидно, что чем больше p_z, тем быстрее частица достигнет плоскости детектора. При $p_z = 0$ частица не долетит до детектора никогда.

С помощью полученных координат x, y можно определить второй параметр вектора признаков – номер пада.
Микроканальный пластинчатый детектор разбит на кольца и сектора. Каждому паду соответствует уникальная пара индексов – номер кольца и номер сектора, что позволяет однозначно идентифицировать область регистрации сигнала.
Для определения номера пада необходимо перейти от декартовой системы координат (x, y) к полярной системе координат (r, φ), где $r -$ это расстояние от центра детектора до точки попадания частицы на детектор, а $\varphi -$ это азимутальный угол, отсчитываемый от оси Ox.
Математически переход задается выражением: $r = \sqrt{x^2 + y^2}, \varphi = \arctan\left(\frac{y}{x}\right)$.
Прежде чем определить номер кольца и сектора, необходимо провести дискретизацию пространства: поделить весь диапазон радиусов и углов на равномерные интервалы (радиус делится на кольцевые зоны, а угол – на секторы), а затем посмотреть в какой промежуток попадает значение r и φ. В результате получаем пару чисел, которая уникальным образом идентифицирует каждый пад: (i_{ring}, i_{sector}). Если частица не попала на чувствительную поверхность детектора, то параметры частицы в векторе признаков заполняются нулями.

Как уже было замечено ранее, важнейшим физическим параметром, регистрируемым в эксперименте, является время пролета – tof, которое рассчитывается как:

$$tof = \frac{z_0}{v}, \text{ где } v = \frac{p}{energy_{csm}} \cdot c$$

Здесь $p = \sqrt{p_x^2 + p_y^2 + p_z^2}$ – полный импульс частицы, $energy_{csm} -$ энергия каждой частицы по отдельности, а $\mathrm{c} = 29.98 \frac{\text{см}}{\text{нс}}$ – скорость света.
Для более удобной физической интерпретации процесса вводится следующий параметр:

$$T = \frac{z_0}{tof - t_0},$$

где $t_o = \frac{z_0}{c} -$ это время, за которое свет достигает поверхности детектора.
То есть частицы, летящие почти со скоростью света (легкие и быстрые), будут иметь очень большие значения T. И, наоборот, частицы с низкими скоростями и, как следствие, большой величиной tof будут иметь маленькое значение T. Это важная характеристика процесса, которая может помочь в дальнейшем анализе события, определив тип частицы.

Таким образом, на вход нейронной сети будет подаваться вектор признаков размера $[3, 120]$, где 120 – это максимальное количество частиц, которые могут быть зафиксированы двумя детекторами одновременно ($2 \cdot rings \cdot sectors = 120$), а 3 – это признаки, характеризующие каждую частицу по отдельности (T, pad number, detector ID).

Следующим этапом работы, после определения структуры вектора признаков, стала подготовка модельных данных для дальнейшего анализа зависимости $\sqrt{s_{eff}}(b)$ и построения функции, способной отобразить сигнал с детектора на выделившуюся после столкновения энергию. Обработка данных (как и все дальнейшие этапы работы), полученных с генератора событий QGSM, моделирующего столкновения тяжелых ионов, была реализована на языке программирования Python. Изначально работа велась с 10 файлами, в каждом по 2000 событий. В процессе работы из-за переобучения модели было принято решение увеличить количество данных до 100 файлов (всего 200 000 событий). Это повысило качество обучения нейронной сети на тренировочных наборах данных, а также помогло модели достичь более точных результатов предсказаний на тестовых выборках.

В ходе работы также использовался ряд специализированных библиотек:

- NumPy [6] – использовалась для работы с многомерными массивами, а также для эффективного и быстрого проведения математических вычислений
- Pandas [7] – применялась для чтения и структурирования файлов в форматах csv и txt
- Matplotlib [8] - обеспечивала визуализацию получаемых результатов
- PyTorch [9] – являлась основной платформой для построения, обучения и оценки нейросетевой модели
- Scipy [10] – использовалась для работы с распределениями и построения функций аппроксимации при анализе статистических свойств ошибок
- Scikit-learn [11] – применялась для разделения данных на тренировочную и тестовую выборки, а также для вычисления метрик

```
  1   10  14.400   2.781  14.129
 1   0   1   0         321   7.4161E-01 7.6406E-01-4.2361E-01 4.7175E+00 4.2361E-01 4.93670E-01
 0   0   0   0         111  -2.2660E-01 5.0864E-02 2.1669E-01 3.2592E+00-2.1669E-01 1.34960E-01
 0   0  -1   0         130   2.6345E-01-3.8472E-01 4.0930E-01 6.9840E+00-4.0930E-01 4.97670E-01
-1   0   0   0        -211   4.9467E-02 6.9877E-02 8.8307E-02 1.5900E+00-8.8307E-02 1.39570E-01
 1   0   0   1        2212   4.0255E-02-4.6151E-01 3.4456E+00 4.0942E+01-3.4456E+00 9.38280E-01
 1   0   0   1        2212  -5.8720E-01 2.6631E-01-4.2285E+00 5.0475E-01 4.2285E+00 9.38280E-01
 1   0   0   1        2212  -2.3434E-01-3.3428E-01 6.0368E+00 7.0660E+01-6.0368E+00 9.38280E-01
 0   0   0   1        2112  -2.8048E-02 2.3955E-02 5.3521E+00 6.2678E+01-5.3521E+00 9.39570E-01
77   0   0 194  1000771940  -1.6648E-02 7.9281E-02 1.0519E+03 1.2315E+04-1.0519E+03 1.80677E+02
78   0   0 196  1000781960  -5.8202E-03-5.4375E-02-1.0627E+03-1.6641E+00 1.0627E+03 1.82540E+02
  2 1153   6.668   5.918  -3.073
 0   1   0   0          22  -2.0510E-01-3.7953E-02-2.0267E-01 4.9110E-01 2.0267E-01 0.00000E+00
 0   1   0   0          22  -4.0834E-01 9.1391E-02 3.7699E-01 5.4558E+00-3.7699E-01 0.00000E+00
 0   1   0   0          22  -6.0982E-02 1.8251E-01-1.4767E-01 5.3465E-01 1.4767E-01 0.00000E+00
 0   1   0   0          22   2.3742E-01 5.0139E-02-5.6060E-01 2.4171E-01 5.6060E-01 0.00000E+00
 0   1   0   0          22  -1.6874E-01 3.3044E-01-2.7939E-01 1.0435E+00 2.7939E-01 0.00000E+00
 0   1   0   0          22   5.0975E-01-2.2054E-01 5.5168E-01 7.7452E+00-5.5168E-01 0.00000E+00
 0   1   0   0          22  -7.6361E-02-8.9971E-02 5.2503E-01 6.1777E+00-5.2503E-01 0.00000E+00
 0   1   0   0          22   3.0057E-01 3.9997E-02 1.1649E+00 1.3763E+01-1.1649E+00 0.00000E+00
 0   1   0   0          22  -9.3515E-02-1.6935E-01 1.5998E-01 2.3846E+00-1.5998E-01 0.00000E+00
-1   1   0   0        -211  -3.5216E-02 9.6279E-02 9.0849E-02 1.3220E+00-9.0849E-02 5.11003E-04
 0   1   0   0          22  -1.4942E-01-1.3503E-01 8.2596E-01 9.7393E+00-8.2596E-01 0.00000E+00
 1   1   0   0         211  -1.8900E-01 2.6198E-01 5.3703E-01 6.7588E+00-5.3703E-01 5.11003E-04
 0   1   0   0          22   1.3617E-01-5.5119E-02 2.7538E-02 1.0234E+00-2.7538E-02 0.00000E+00
 0   1   0   0          22   1.6661E-01-2.9226E-01 1.5088E+00 1.7749E+01-1.5088E+00 0.00000E+00
 0   1   0   0          22   1.8406E-01-1.6032E-01 6.0495E-01 7.3044E+00-6.0495E-01 0.00000E+00
 0   1   0   0          22  -3.9315E-01-1.6070E-01 1.1448E+00 1.3746E+01-1.1448E+00 0.00000E+00
 0   0   1   0         310  -2.0718E-01 6.1771E-03 1.8449E-01 4.3668E+00-1.8449E-01 4.97670E-01
 0   0   0   0         111  -8.0204E-02 2.9563E-02-2.3577E-01 2.6251E-01 2.3577E-01 1.34960E-01
 0   0   0   0         111   6.7795E-02-8.1357E-03-3.4650E-01 1.5230E-01 3.4650E-01 1.34960E-01
 0   0   0   0         111   3.8335E-02-3.2636E-02-5.1528E-02 5.8079E-01 5.1528E-02 1.34960E-01
```

Рис. 2: Исходный вид модельных данных генератора QGSM

Каждый файл представляет собой структурированный массив, в котором данные записаны пособытийно. Каждое столкновение происходит при определенном прицельном параметре. В начале каждого события указывается номер события, количество частиц, прицельный параметр и его проекции на оси Ox и Oy: $b = \sqrt{b_x^2 + b_y^2}$.
Далее построчно идут характеристики частиц. В ходе работы активно использовались следующие величины: заряд частицы (первый столбец), pdg-код частицы (пятый столбец), проекции импульса в системе центра масс p_x, p_y, p_z (шестой, седьмой, восьмой столбцы), m масса частицы (одиннадцатый столбец).
Одним из важнейших этапов обработки данных является этап отделения спектаторов от

остальных частиц. Спектатор – это частица, которая была частью ядра еще до столкновения, практически не изменила траекторию своего движения, а также энергию. Эти частицы не несут информацию о самом процессе взаимодействия, и их исключение позволяет оценить энергию частиц, участвовавших во взаимодействии. Будем называть эту величину выделившейся энергией.

Были реализованы два фильтра по отбору спектаторов от остальных частиц:

1. Энергетический фильтр:

$$energy_{csm} \approx \sqrt{s_0}$$ – условие равенства энергий до и после столкновения

2. Фильтр по импульсу:

$$\left(p_x^2 + p_y^2\right) \ll p_z^2$$ – условие на малость поперечной компоненты импульса по сравнению с продольной

Также в ходе работы было реализовано два подхода в подготовке данных для обучающей и тестовой выборок:

1. Использование всех частиц, независимо от их заряда. В этом случае охватывается максимально широкий спектр событий, и энергия, выделившаяся в ходе столкновения, смотрится для всех частиц, образовавшихся в ходе столкновения
2. Отбор только заряженных частиц. Этот подход обоснован тем, что реальные экспериментальные установки (например детекторы на микроканальных пластинах) способны регистрировать в основном только заряженные частицы. Таким образом мы можем приближенно воспроизвести реальные экспериментальные условия.

Сравнение работы модели в этих двух случаях поможет оценить степень влияния нейтральных частиц на итоговый результат (точность предсказания выделившейся в ходе столкновения энергии) и, как следствие, определить какой набор данных лучше подойдет для формирования вектора признаков.

Перейдем теперь непосредственно к формулировке задачи регрессии и описанию процесса обучения модели.

В данной работе мы стремимся обучить модель таким образом, чтобы она могла с высокой точностью предсказывать скалярную величину – энергию, выделившуюся в результате ядерного столкновения. То есть мы ставим перед собой задачу научить нейронную сеть производить численную оценку исследуемой величины. Наша модель должна получать на вход вектор признаков, а на выходе выдавать только одно число – выделившуюся в событии энергию. Обучение модели проводится на размеченных данных, где для каждого входного вектора признаков ставится в соответствие значение выделившейся при столкновении энергии.

Один цикл обучения нейросетевой модели называется эпохой (epoch) – за одну эпоху весь набор тренировочных данных должен пройти через алгоритм. Для повышения устойчивости процесса к переобучению и шумам обучающая выборка предварительно делится на пакеты (batch) (выборки фиксированного и маленького размера), формируемые случайным образом. Каждый пакет подается на вход нейронной сети и поэтапно обрабатывается: модель на основании входного вектора признаков формирует предсказание энергии столкновения, а затем вычисляется значение функции ошибки – в данной работе это MSE. После расчета

ошибки запускается механизм обратного распространения ошибки (backpropagation), в ходе которого находятся градиенты весов нейронной сети. Эти градиенты используются для дальнейшей корректировки весов сети при помощи оптимизатора $Adam$, автоматически подбирающего оптимальную скорость обучения. После обновления параметров градиенты обнуляются, а процедура повторяется заново для оставшихся пакетов из тренировочной выборки.

Описанный выше процесс повторяется до тех пор, пока не завершается последняя эпоха. В конце модель проходит тестирование на тестовой выборке, состоящей из данных, ранее не виденных моделью. В ходе данного процесса оценивается способность модели к обобщению исследуемых зависимостей, а также ее умение строить точные предсказания.

В ходе данной работы были рассмотрены две модели нейронной сети: полносвязная НС и нейронная сеть с одним сверточным слоем.

1. Полносвязная нейронная сеть (fully connected neural network):

Каждое линейное преобразование модели имеет следующий вид:

$$\vec{t}^{(0)} = \vec{x}, \qquad \vec{t}^{(i+1)} = \sigma\left(W_i \vec{t}^{(i)} + \vec{b}^{(i)}\right), \qquad i = 0, \dots, N$$

- $\vec{x} \in \mathbb{R}^{360}$ –входной вектор признаков,
- $\vec{t}^{(i)} \in \mathbb{R}^{N_i}$ – выход i-ого слоя,
- $W_i \in \mathbb{R}^{N_{i+1} \times N_i}$ – матрица весов слоя,
- $\vec{b}^{(i)} \in \mathbb{R}^{N_{i+1}}$ – вектор смещений,
- $\sigma(\cdot)$ – функция активации SiLU, где $\sigma(x) = x \cdot \frac{1}{1+e^{-x}}$,

В данной архитектуре используется $N = 3$ скрытых слоя. Выходной слой выглядит следующим образом:

$$\hat{y} = W_{N+1} \vec{t}^{(N+1)} + \vec{b}^{(N+1)}$$

Также рассмотрим таблицу гиперпараметров, справедливую как для этой архитектуры нейронной сети, так и для следующей:

Таблица 1: Гиперпараметры полносвязной нейронной сети

Activation function	Loss function	Optimizator	Weight decay	Learning rate	Batch size	Epochs
SiLU	MSE	Adam	10^{-5}	0.001	3000	1500

2. Сеть со сверточным слоем (convolutional layer):

Сверточный слой применяет операцию свертки к входному тензору $\boldsymbol{X} \in \mathbb{R}^{1 \times H_{in} \times W_{in}}$, где 1 входной канал, H_{in} – это высота, а W_{in} – это ширина. В рассматриваемой работе $H_{in} = 120$, а $W_{in} = 3$ – для каждой частицы рассматриваем вектор признаков (tof, pad number, detector ID). Входной тензор $\boldsymbol{X}$ в работе обрабатывается двумерной сверткой с 8 фильтрами и ядром размерности 6×3 (фильтром проходим по 6 частицам за раз, а на каждую частицу берем вектор признаков). Это преобразование можно записать в следующем виде:

$$\boldsymbol{F}_k = ReLU\left(\sum_{c=1}^{I} \boldsymbol{X}_c * \boldsymbol{K}_{k,c} + b_k\right), \qquad k = 1, \dots, K$$

$I -$ это количество входных каналов ($I = 1$), $K -$ это количество фильтров ($K = 8$), $\boldsymbol{K}_{k,c} \in \mathbb{R}^{6\times3} -$ сверточные ядра, $\boldsymbol{X}_c \in \mathbb{R}^{120\times3} -$ это c-ый входной канал, $* -$ это операция двумерной свертки, $\boldsymbol{F}_k \in \mathbb{R}^{H_{out}\times W_{out}} -$ это результат применения k –ого фильтра, ReLU – функция активации. Посчитаем значения H_{out} и W_{out} по следующей формуле:

$output_{size} = input_{size} - kernel_{size} + 1$. Тогда на выходе получим, что:

$H_{out} = 120 - 6 + 1 = 115, W_{out} = 3 - 3 + 1 = 1 => \boldsymbol{F}_k \in \mathbb{R}^{115\times1}$

После применения всех 8-ми фильтров мы получаем тензор $\boldsymbol{F} \in \mathbb{R}^{K\times H_{out}\times W_{out}}$ ($\mathbb{R}^{8\times115\times1}$), который затем при помощи функции flatten выравнивается в вектор $\vec{f} \in \mathbb{R}^{1840}$. Операция flattening необходима для того, чтобы преобразовать тензор, получаемый после свертки, к виду, который можно подать на вход нейронной сети.

Далее вектор $\vec{f}$ последовательно проходит через ряд полносвязных слоев, уже рассмотренных для первой модели.

4. Проверка выполнения ЗСЭ и ЗСИ

Из теоретической справки мы знаем значение энергии до столкновения:

$$E_0 = \sqrt{s_{total}} = 2168\ GeV$$

Следовательно, для проверки выполнения ЗСЭ необходимо подсчитать энергию после столкновения – сумму энергий спектаторов и образовавшихся частиц. Имеем модельные данные, взятые из генератора QGSM (pdg – код частицы, масса частицы, компоненты импульса в системе центра масс). Воспользуемся формулой (2) для подсчёта $E_1 = \sum energy_{csm}$.

Введём величину:

$$\eta = \frac{E_1 - E_0}{E_0},$$

где $\eta -$ это относительное отклонение величины E_1 от E_0. Значение E_0 есть ожидаемое среднее значение величины E_1 (усреднение по различным прицельным параметрам), следовательно, выполнения ЗСЭ эквивалентно малости среднего значения η.

Построим гистограмму, откладывая по оси Ox значение η, а по оси Oy число попаданий в данный промежуток среди всевозможных значений прицельного параметра (у нас их конечное число – 2000 точек).

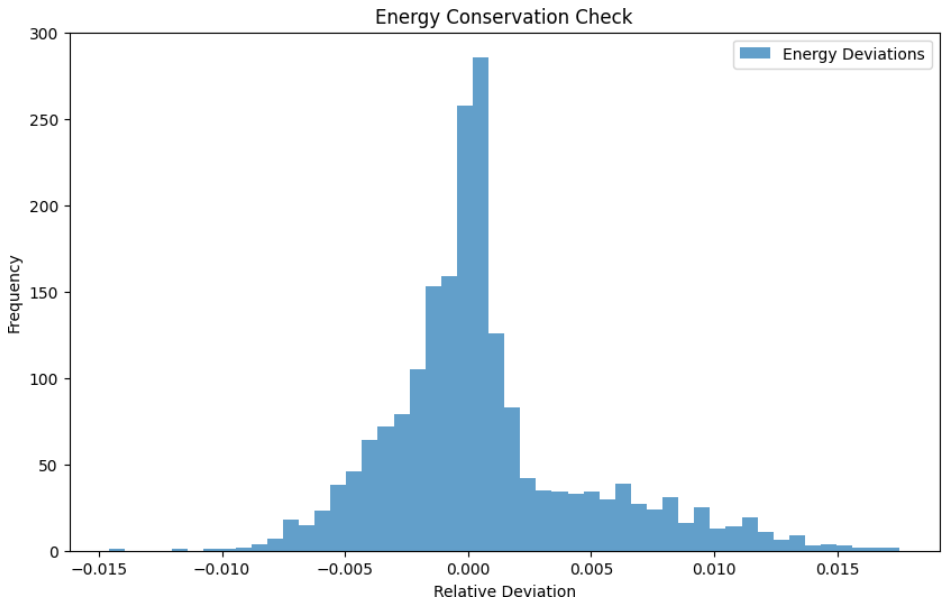


Рис. 1: Распределение событий в зависимости от η

Как видно из гистограммы на Рис. 1, основная часть событий сосредоточена в непосредственной окрестности нуля, что указывает на выполнение закона сохранения энергии с высокой точностью. Распределение относительно η имеет четко выраженный пик около нулевого значения, а также симметрично убывающие “хвосты”.
Обратим также внимание, что почти все точки лежат в промежутке [-0.015; 0.015]. То есть ЗС для энергии выполняется в пределах 1,5%.

Теперь проверим выполнение закона сохранения импульса для каждой компоненты импульса по отдельности. Для проверки ЗСИ нам нужно, чтобы выполнялось условие:
$\vec{p}_{initial} = \Sigma_i \vec{p}_j^i$, где $j = x, y, z$, а i – это конкретное событие ($i \in [1; 2000]$), а $\vec{p}_{initial} = 0$.
Это значение импульса в начальный момент времени (до столкновения) в системе центра масс. Построим графики для p_x, p_y, p_z:

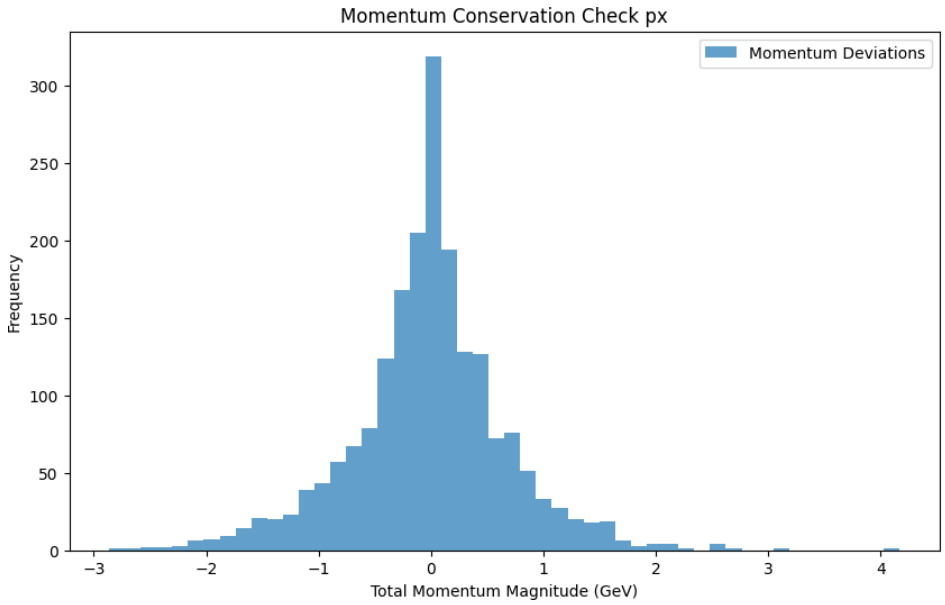


Рис. 2: Распределение частиц по компоненте импульса p_x

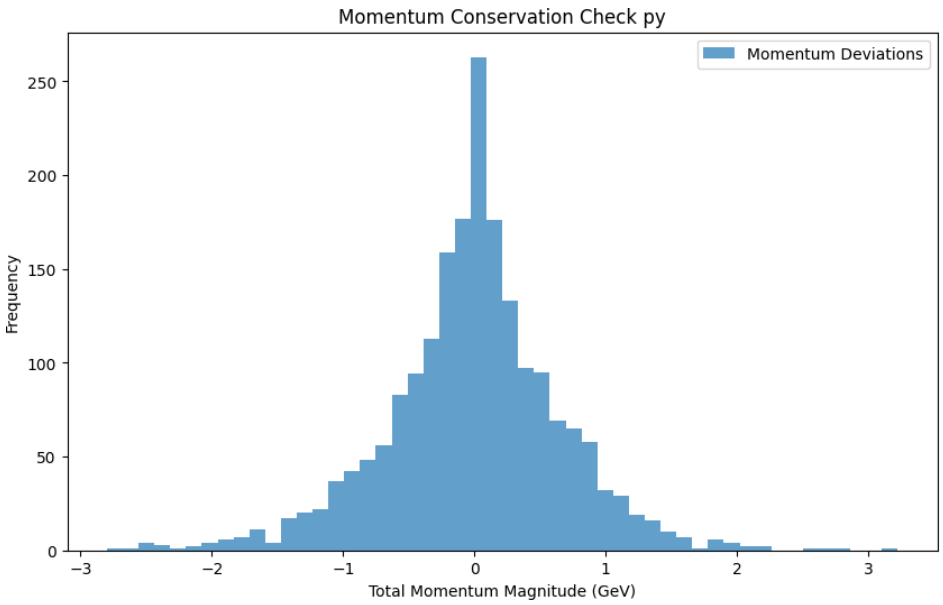


Рис. 3: Распределение частиц по компоненте импульса p_y

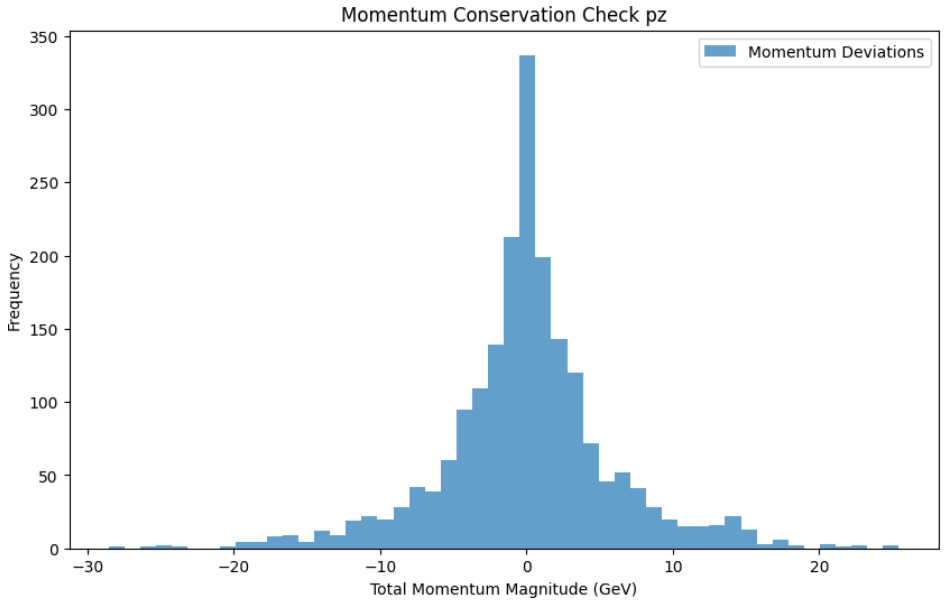


Рис. 4: Распределение частиц по компоненте импульса p_z

Из графиков видно, что ЗСИ выполняется лучше для компонент p_x и p_y.

5. Фильтры энергии и импульса

По используемому определению, спектаторами не могут являться частицы, образующиеся после столкновения. Исходя из этого, воспользуемся сортировкой по частицам, то есть будем брать только нуклоны и тяжелые ядра. Спектаторы будем определять двумя способами:

- Энергия до и после столкновения одинаковая
- Поперечная составляющая импульса много меньше продольной

Оба упомянутых метода требуют задания погрешности (в идеальных условиях она стремится к 0%).

Для энергетического способа определения спектаторов погрешностью будем называть значение отклонения от теоретического значения:

$$RelErr = \frac{E_0 - \Delta E}{E_0}$$

Таким образом, частицу считаем спектатором, если она является нуклоном или тяжелым ядром, а также её энергия попадает в следующий промежуток: $[E_0 - \Delta E;\ E_0 + \Delta E]$

Для способа определения спектаторов через компоненты импульса погрешностью будем называть величину:

$$RelErr1 = \frac{\sqrt{p_x^2 + p_y^2}}{|p_z|}$$

Таким образом, частицу считаем спектатором, если она является нуклоном или тяжелым ядром, а также её поперечная составляющая импульса попадает в следующий промежуток: $[0;\ RelErr1 \cdot p_z]$

6. Построение и анализ зависимости энергии взаимодействия от прицельного параметра

Сперва попробуем различные значения погрешностей (выбирая $RelErr = RelErr1$) для обоих фильтров. На графиках синим цветом отображена энергия частиц, образовавшихся в ходе столкновения в зависимости от прицельного параметра, а зеленым – энергия спектаторов от прицельного параметра.

Погрешность 6%:

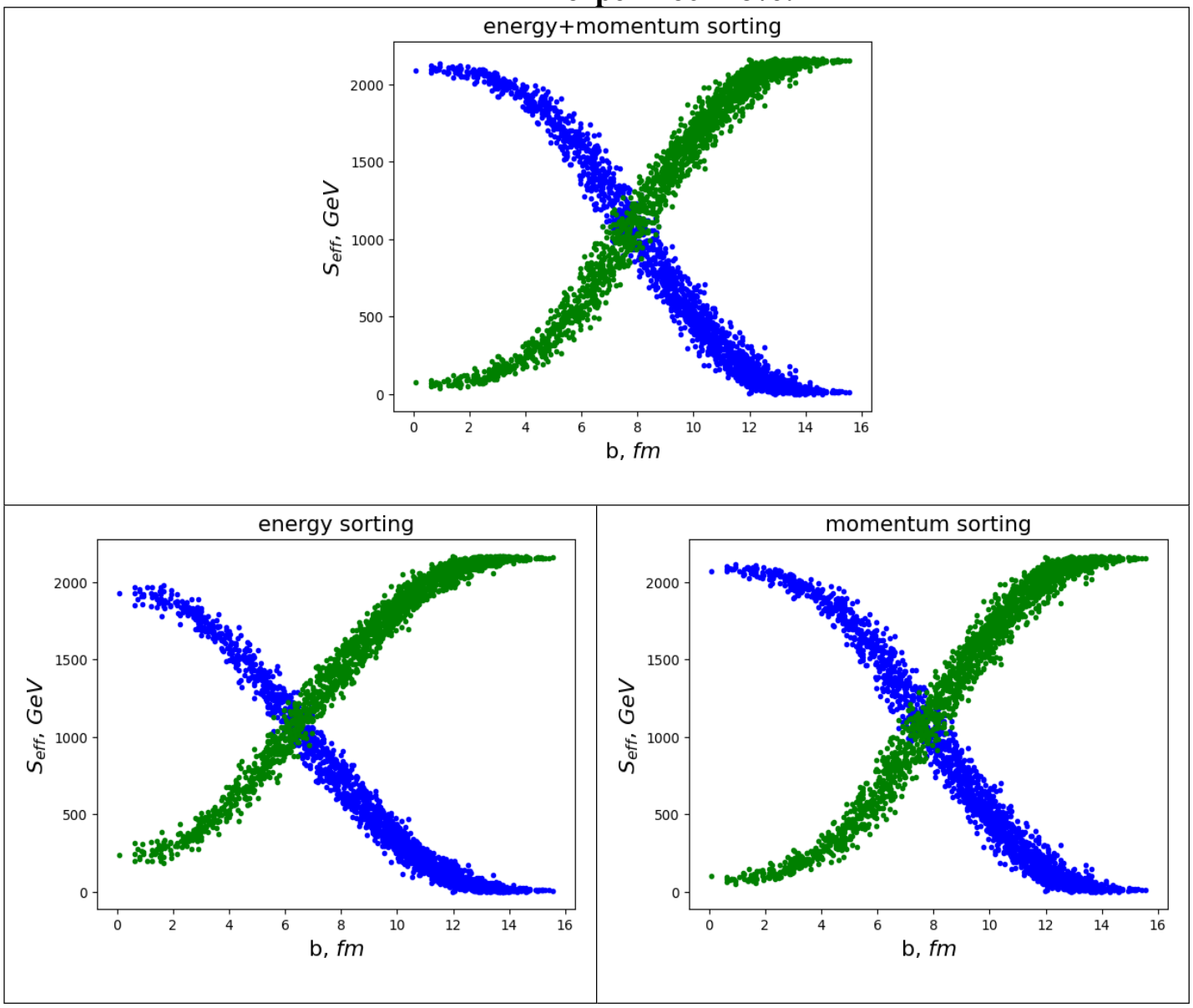


Рис. 5: Зависимость $\sqrt{s_{eff}}\,(b)$ при $Err = 6\%$. Синим цветом отображается энергия частиц, образовавшихся в ходе столкновения, от прицельного параметра, зеленым – энергия спектаторов от прицельного параметра.

Погрешность 10%:

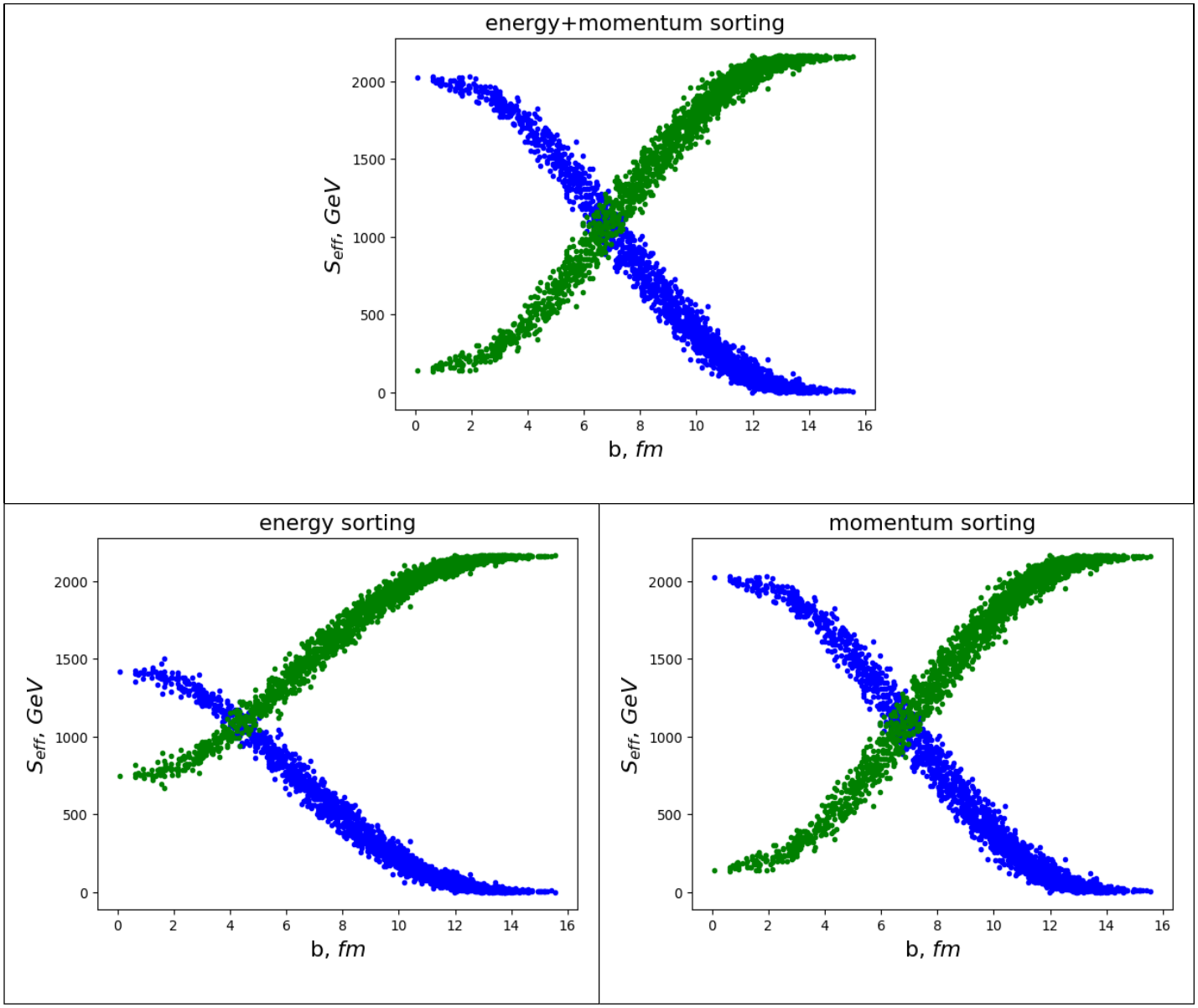


Рис. 6: Зависимость $\sqrt{s_{eff}}\,(b)$ при $Err = 10\%$. Синим цветом отображается энергия частиц, образовавшихся в ходе столкновения, от прицельного параметра, зеленым – энергия спектаторов от прицельного параметра.

При погрешности 6% и более видно, что графики, получающиеся при использовании фильтра отбора спектаторов по энергиям, сильно меняются: при лобовом столкновении ($b = 0$), энергии спектаторов не равны нулю, а это противоречит теории.

Попробуем взять теперь очень маленькую погрешность: $Err = 0.1\%$

Погрешность 0,1%:

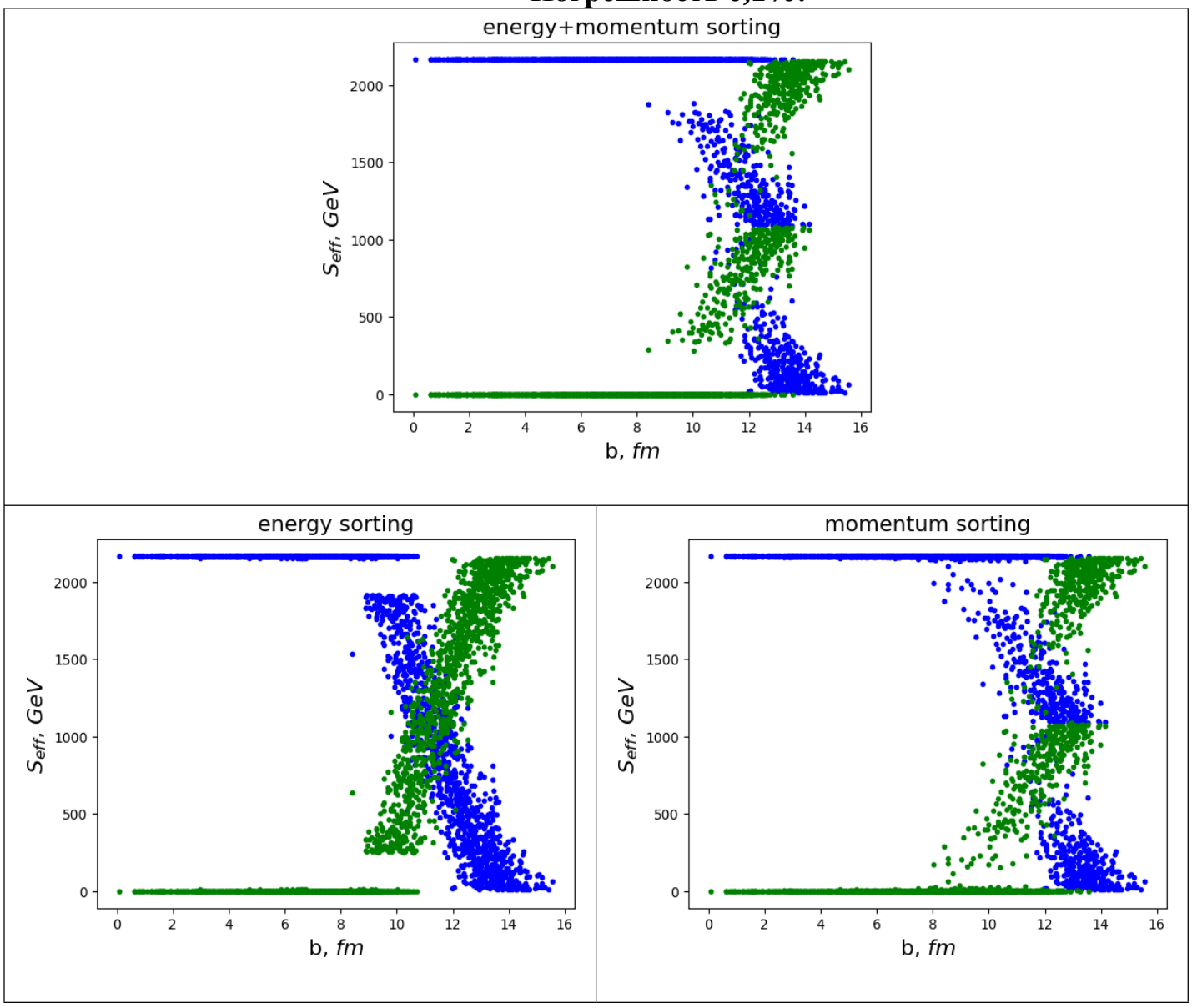


Рис. 7: Зависимость $\sqrt{s_{eff}}\,(b)$ при $Err = 0.1\%$. Синим цветом отображается энергия частиц, образовавшихся в ходе столкновения, от прицельного параметра, зеленым – энергия спектаторов от прицельного параметра.

Как видно из графика, полученные зависимости не интерпретируются физически и не отражают ожидаемой зависимости $\sqrt{s_{eff}}(b)$. Подобное поведение может быть обусловлено особенностями используемых алгоритмов в генераторе QGSM.

Из законов сохранения мы заметили, что основная часть точек для энергий лежит в интервале 1.5%, а для импульса в интервале 14%. При учете погрешности 14% у нас получаются графики, плохо интерпретируемые физически. Это может быть связано с неточностью модельных данных. Перебирая различные погрешности для фильтра по импульсу, было выяснено, что при погрешности 2% получаются наилучшие графики. Построим с учетом таких погрешностей графики зависимости энергий от прицельного параметра.

Погрешность 1.5 % по энергии и 2% по импульсу:

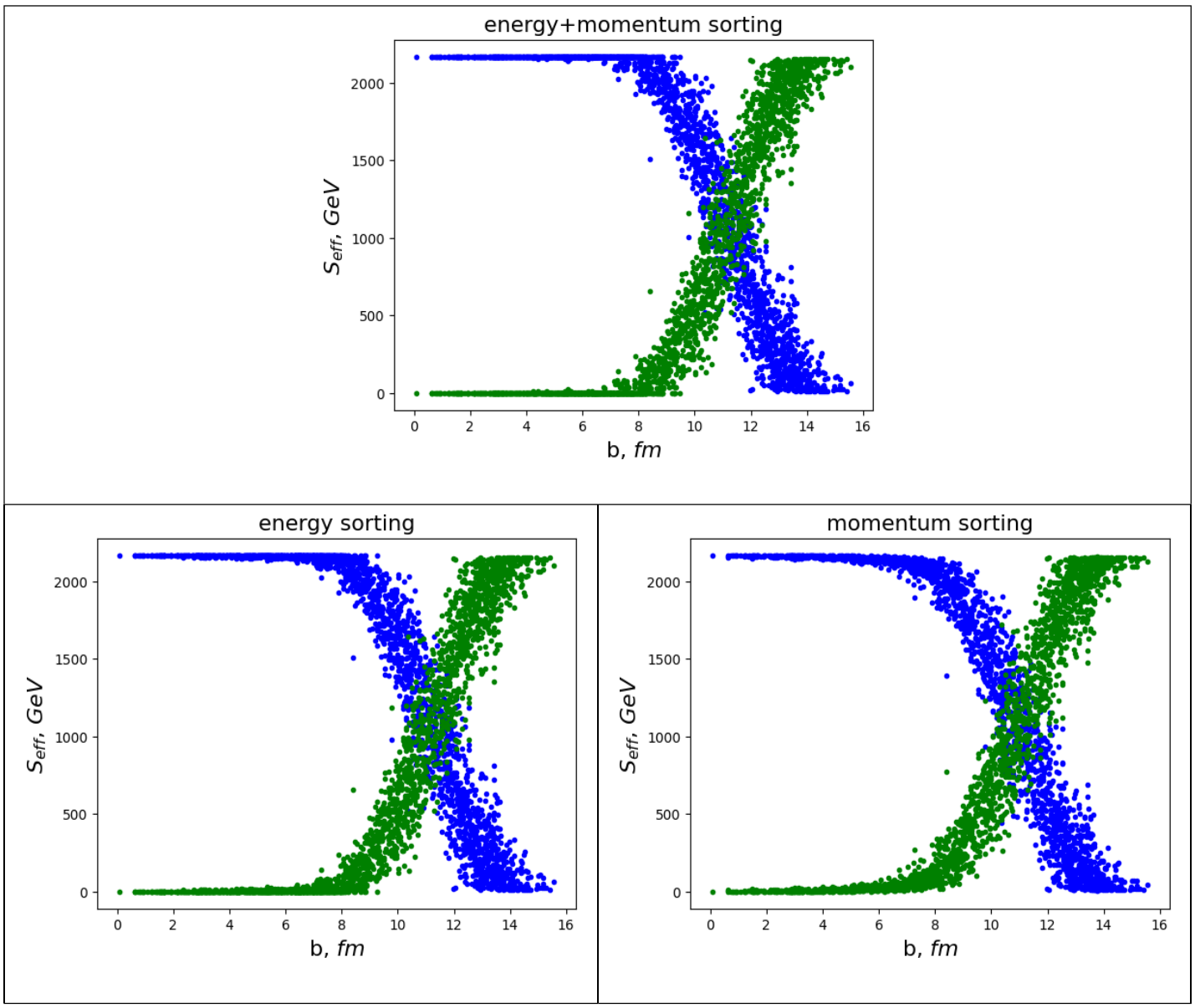


Рис. 8: Зависимость $\sqrt{s_{eff}}\,(b)$ при $Err = 1.5\%$ и 2%. Синим цветом отображается энергия частиц, образовавшихся в ходе столкновения, от прицельного параметра, зеленым – энергия спектаторов от прицельного параметра.

Как видно, графики по своему виду совпадают с ожидаемыми: энергия спектаторов быстро растет при прицельном параметре порядка размера ядра.

7. Построение функции, отображающей сигнал с детектора на выделившуюся энергию

На завершающем этапе работы были построены и обучены две нейросетевые модели (полносвязная НС, а также НС с добавлением одного сверточного слоя), задача которых – восстановление энергии, выделившейся в ходе столкновения двух ядер золота, на основе сигнала, полученного с микроканальных пластинчатых детекторов. Ниже приведены результаты работы обеих моделей для двух различных случаев: когда детектор способен фиксировать только заряженные частицы и когда детектор фиксирует абсолютно все частицы, долетевшие до него.

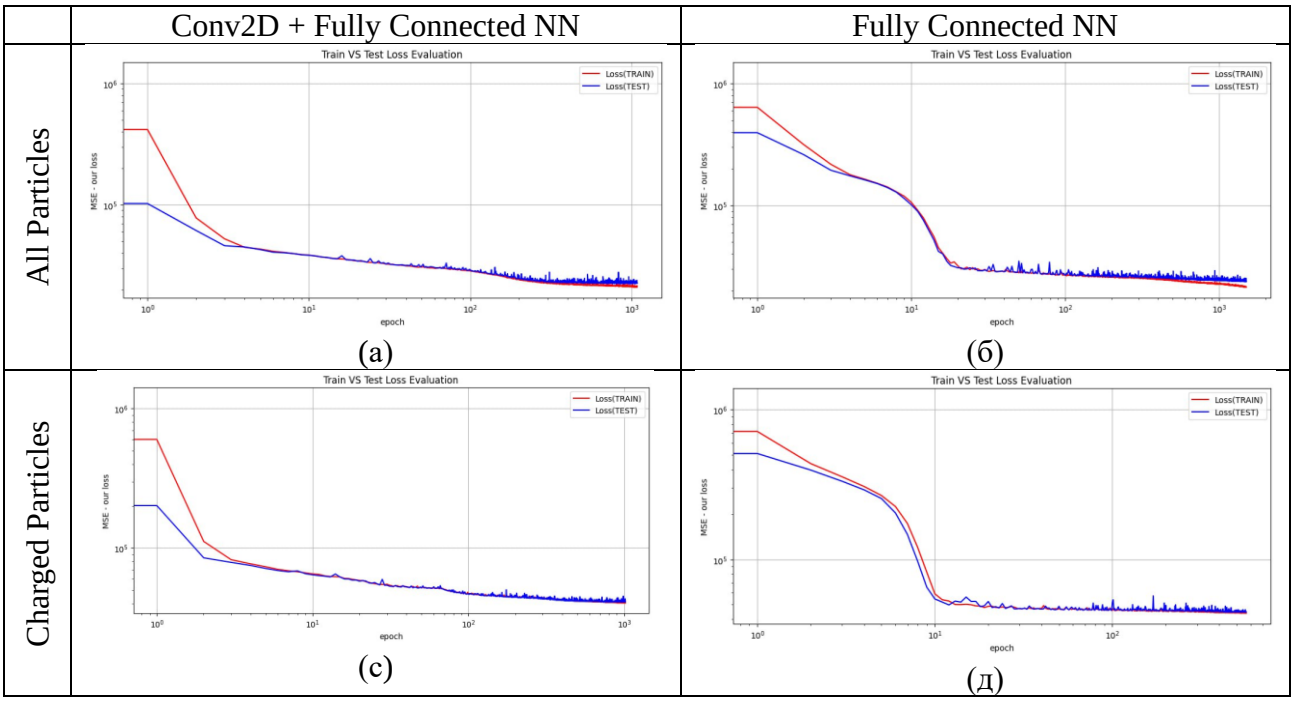
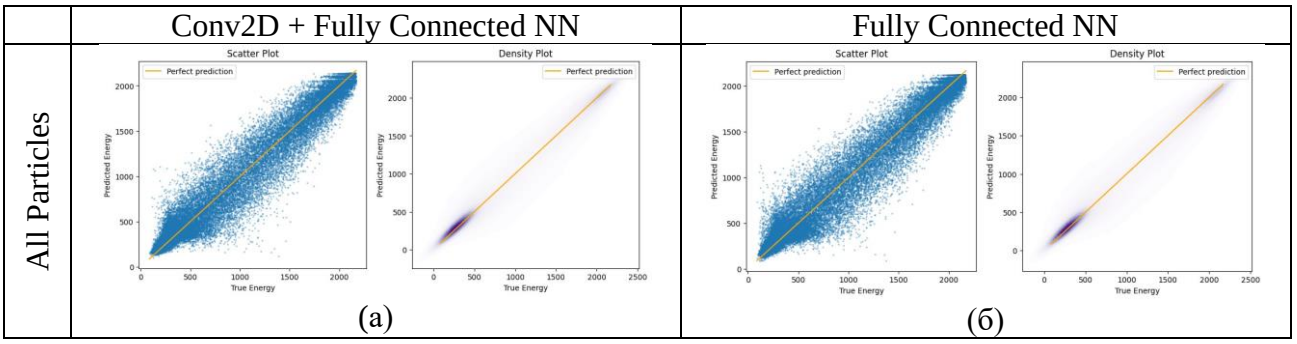


Рис. 9: Эволюция функции потерь со временем на тестовой и тренировочной выборках: а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.



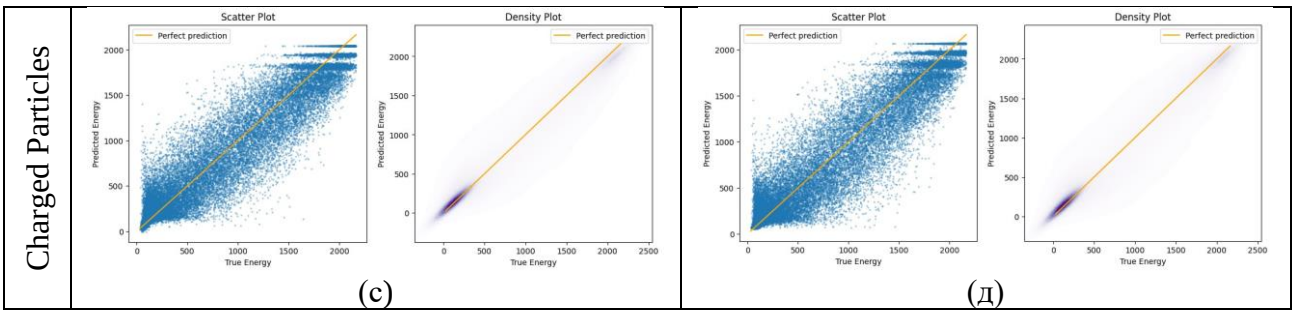


Рис. 10: Зависимость предсказанной энергии от истинной, а также график плотности распределения энергий: а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.

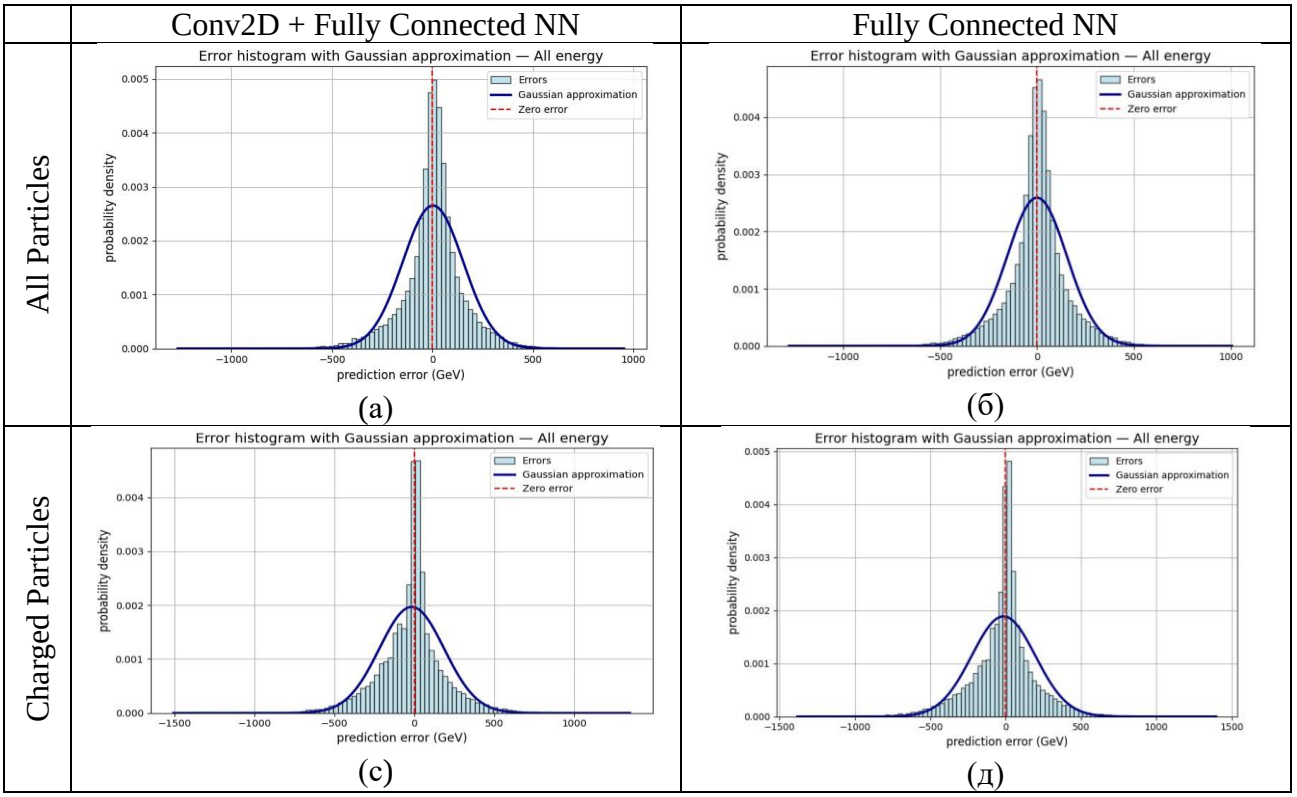


Рис. 11: Гистограмма распределения ошибки предсказания с Гауссовой аппроксимацией: а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.

На Рис. 9 приведены графики изменения функции ошибки во время обучения на тренировочной и тестовой выборках. Графики показывают, как ведет себя модель с увеличением числа эпох, и позволяют судить о сходимости и обобщающей способности нейронной сети. Для количественной оценки качества восстановления энергии использовалась среднеквадратичная ошибка RMSE, задающаяся формулой:

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\widehat{y_i} - y_i)^2},$$

где:

- $\widehat{y_i}$ – это предсказанное моделью значение энергии,
- y_i – это истинное значение энергии,
- N – это количество событий в тестовой выборке.

На основе этой метрики можно сравнить результаты работы двух моделей для двух разных случаев. Ниже приведены результаты вычисления RMSE:

Таблица 2: Значения RMSE в GeV

	Conv2D + Fully Connected NN	Fully Connected
All Particles	150.5	154.0
Charged Particles	203.3	211.6

Как видно из Таблицы 2, когда модель обучается на всех частицах, включая нейтральные, нейронная сеть показывает более высокую точность. Хотя в реальных условиях микроканальные пластинчатые детекторы регистрируют только заряженные частицы, учет всех частиц в ходе обучения – это теоретическая модель, которая позволяет нейронной сети лучше понять связь энергии, выделившейся в событии, с вектором признаком.

Для оценки качества предсказаний помимо абсолютного значения RMSE рассчитаем относительную погрешность по формуле:

$$\varepsilon = \frac{RMSE}{\sqrt{s_{total}}} \cdot 100\%$$

Таблица 3: Относительная ошибка ε

	Conv2D + Fully Connected NN	Fully Connected
All Particles	6.95%	7.10%
Charged Particles	9.38%	9.77%

Полученные значения относительной погрешности показывают, что минимальная погрешность достигается при использовании для обучения нейронной сети всех частиц в модели с добавлением сверточного слоя - $\varepsilon = 6.95\%$. Это является наилучшим результатом среди всех конфигураций. Максимальная же ошибка $\varepsilon = 9.77\%$ наблюдается для полносвязной нейронной сети, обученной только на заряженных частицах.

Таким образом, можно сделать вывод о том, что использование всех частиц для обучения дает лучший результат, что подтверждаем важность учитывания нейтральных частиц при анализе энергии столкновения. Также архитектура с использованием сверточного слоя в обоих случаях (все частицы и только заряженные) устойчиво дает меньшую ошибку, чем полносвязная НС. При этом значения ε во всех случая находятся в приемлемом диапазоне значений и не превышают 10%, что свидетельствует о том, что модель достаточно хорошо предсказывает интересующую нас величину, даже на основе данных, собранных из весьма малого телесного угла.

Как показали гистограммы распределения ошибки на полной тестовой выборке (весь диапазон энергий, Рис. 11), полученное распределение не совсем хорошо аппроксимируется нормальным распределением. Это особенно заметно по утолщению “хвостов”, а также в области нулевой ошибки. Такие особенности могут быть связаны с тем, что модель с разной точностью справляется с предсказанием величины в разных числовых диапазонах энергий.

Разобьем энергетический диапазон на три равные части, чтобы глубже рассмотреть следующие интервалы: низкоэнергетические события (Low Energy), область средних энергий (Mid Energy), высокоэнергетические энергии (High Energy):

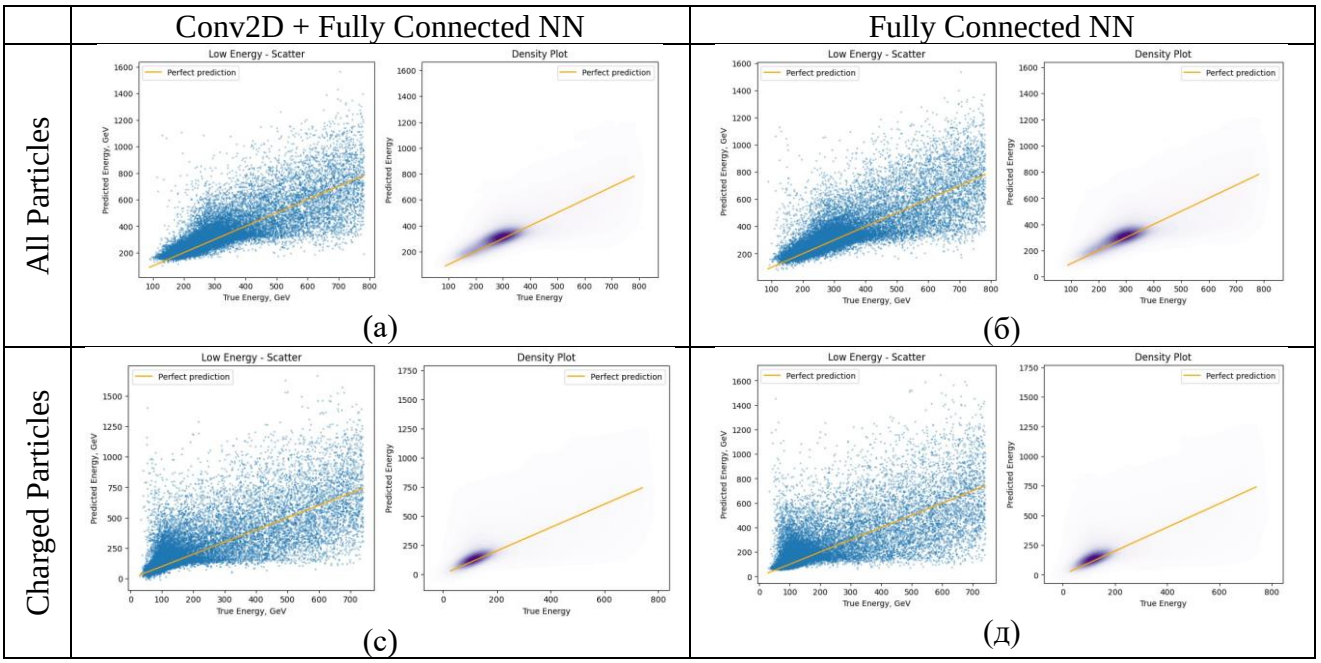


Рис. 12: Зависимость предсказанной энергии от истинной, а также график плотности распределения энергий (Низкоэнергетические столкновения): а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.

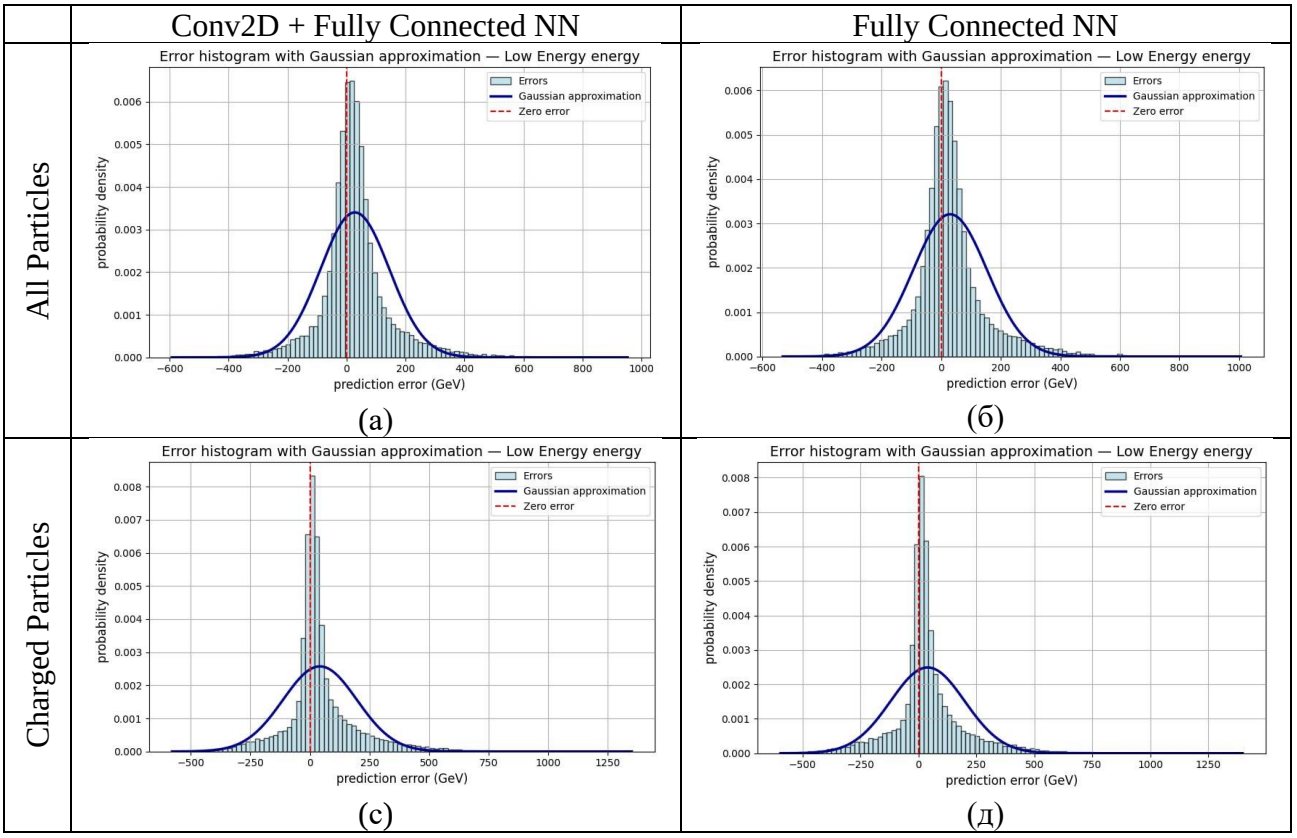


Рис. 13: Гистограмма распределения ошибки предсказания с Гауссовой аппроксимацией (Низкоэнергетические столкновения): а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.

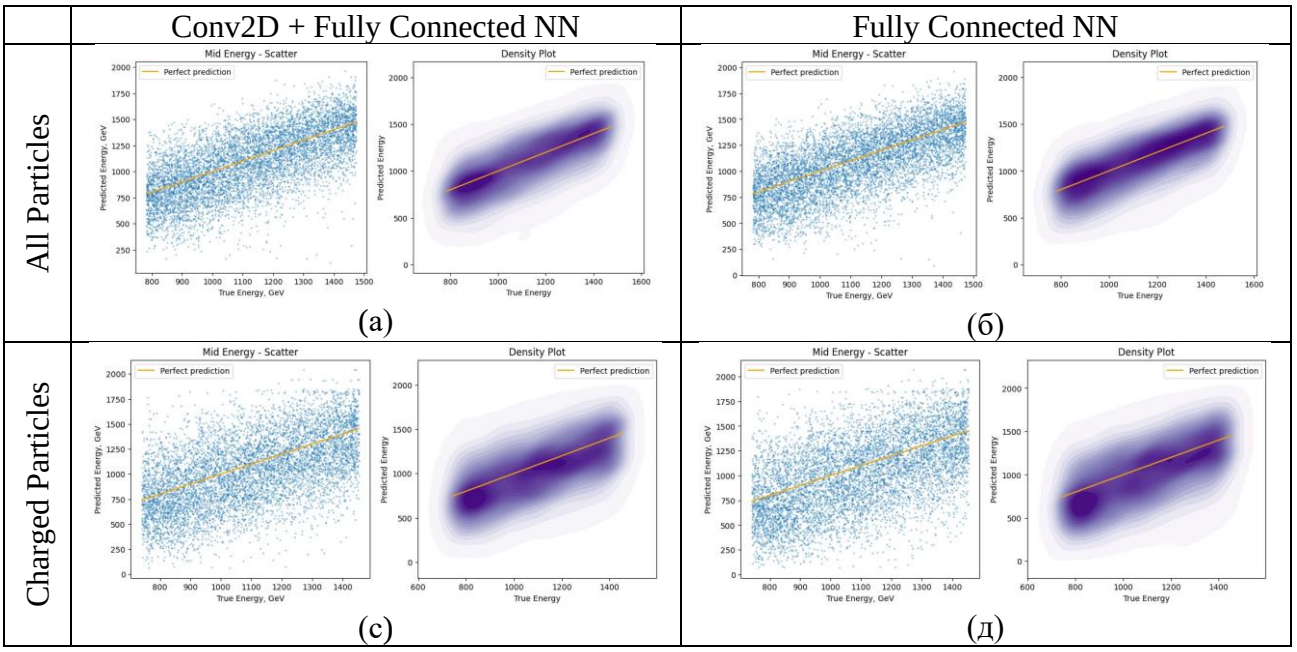


Рис. 14: Зависимость предсказанной энергии от истинной, а также график плотности распределения энергий (Столкновения со средней энергией): а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.

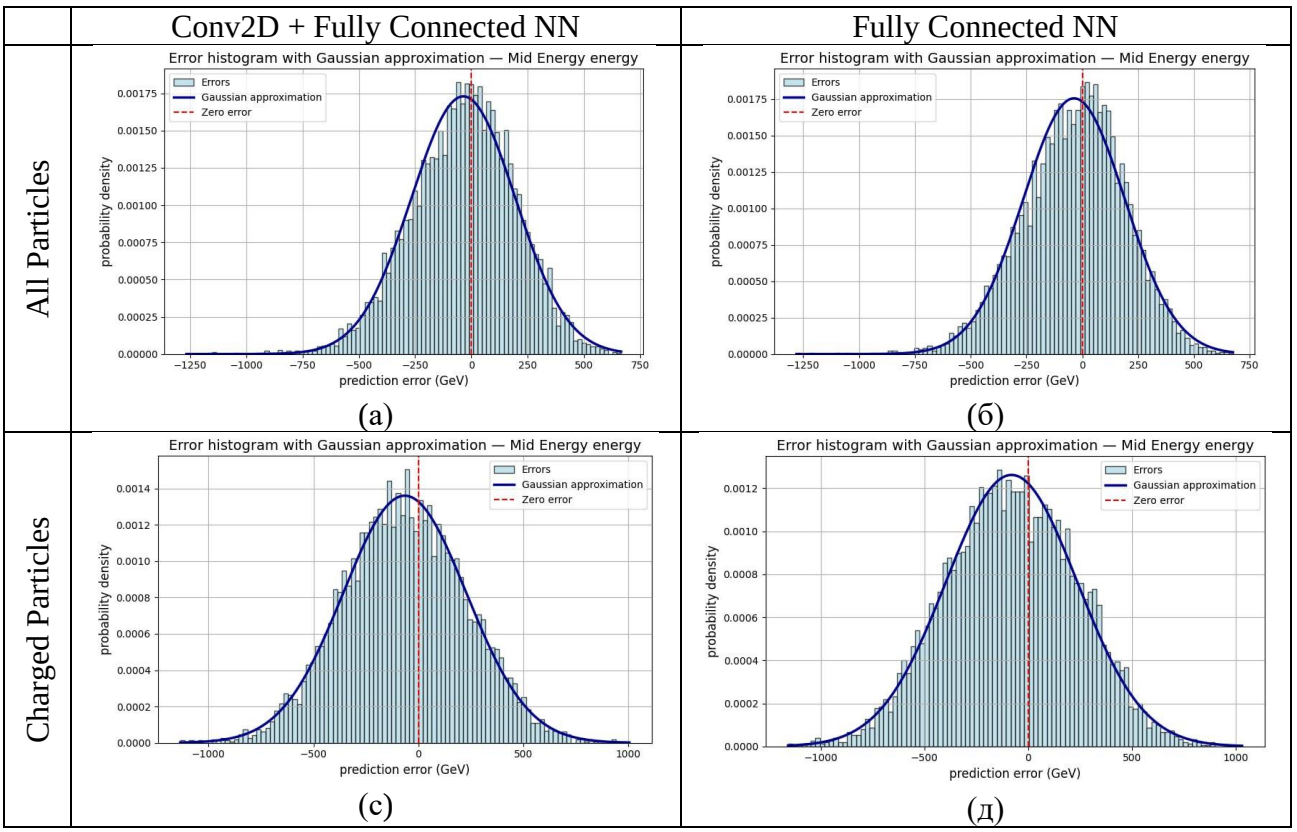


Рис. 15: Гистограмма распределения ошибки предсказания с Гауссовой аппроксимацией

(Столкновения со средней энергией): а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.

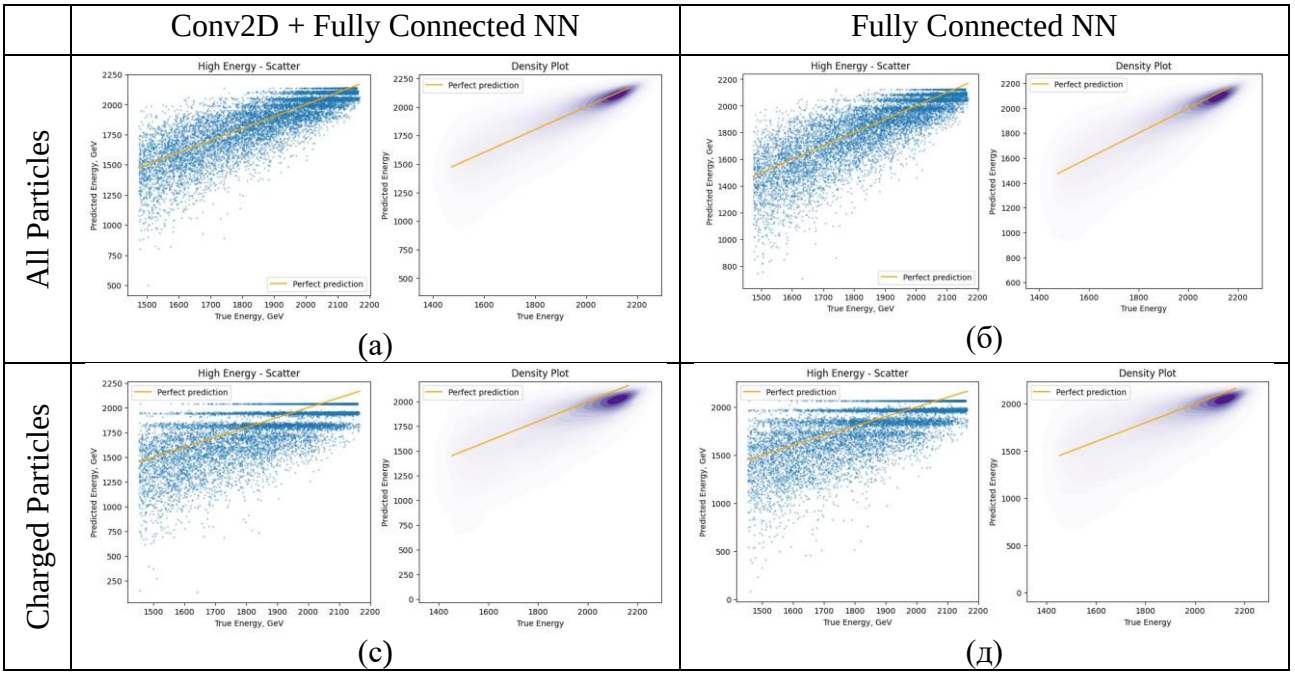


Рис. 16: Зависимость предсказанной энергии от истинной, а также график плотности распределения энергий (Высокоэнергетические столкновения): а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.

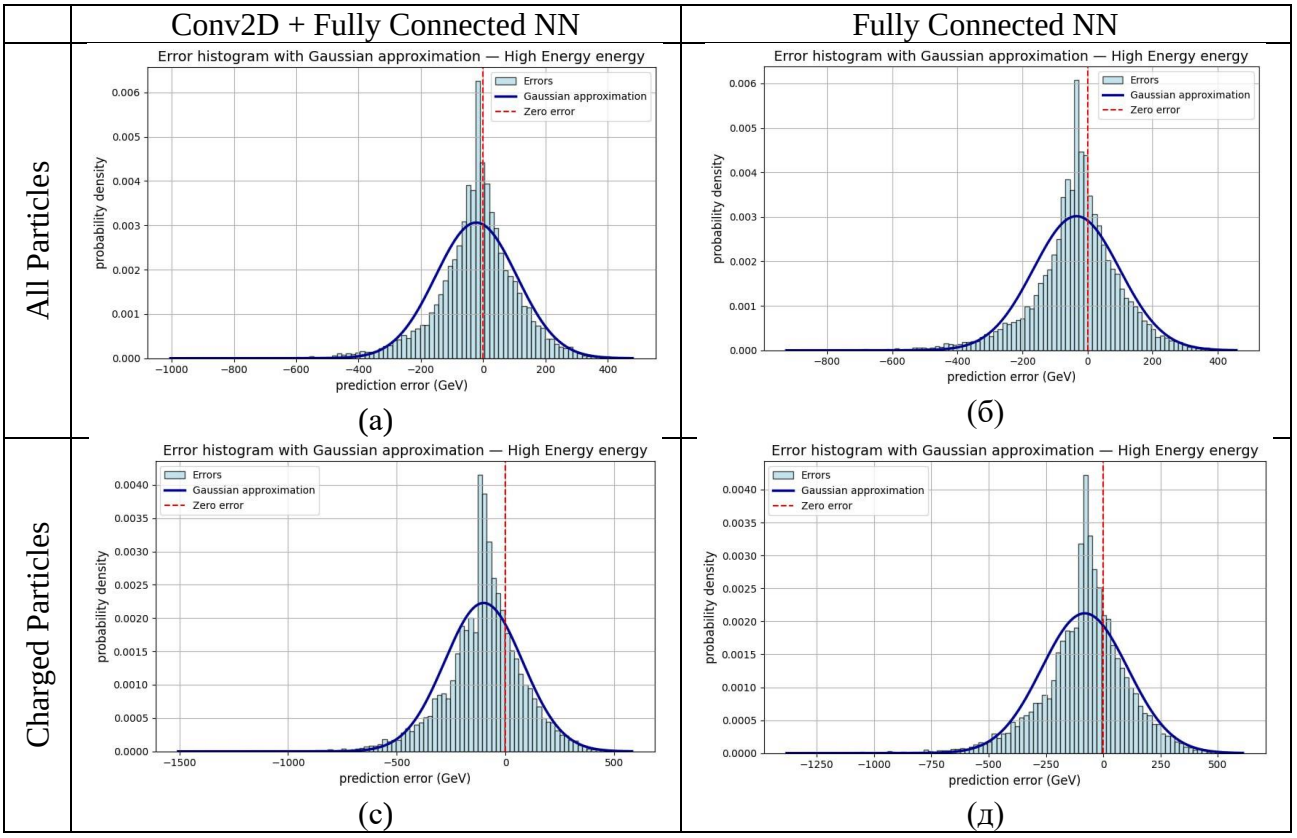


Рис. 17: Гистограмма распределения ошибки предсказания с Гауссовой аппроксимацией (Высокоэнергетические столкновения): а – все частицы и гибридная НС, б – все частицы и полносвязная НС, с – заряженные частицы и гибридная НС, д – заряженные частицы и полносвязная НС.

Распределение ошибки предсказания были проанализированы для трех диапазонов энергий: низкие, средние и высокие. Полученные гистограммы (Рис. 13, Рис. 15, Рис. 17) аппроксимированы гауссианом, что позволяет оценить симметрию, форму и сдвиг ошибки в каждом энергетическом интервале.

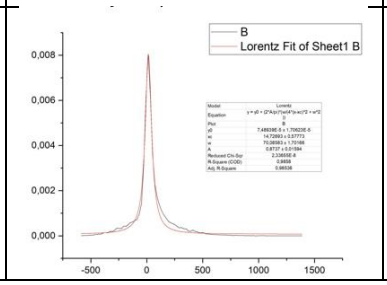
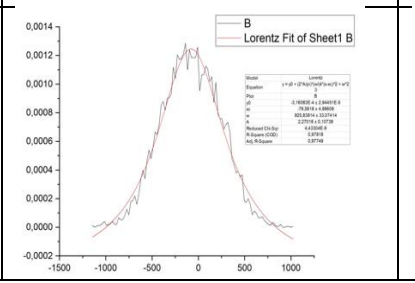
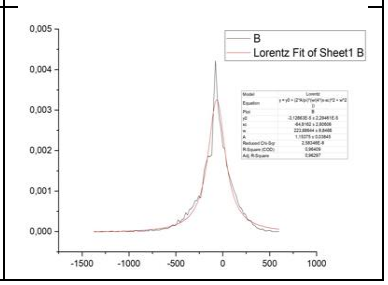
В области средних энергий гистограммы во всех рассматриваемых случаях показывают наиболее симметричное и близкое к нормальному распределение ошибки. Это указывает на то, что модель хорошо обучена на данных из этого диапазона, а также на то, что она стабильно и достаточно точно оценивать выделившуюся в ходе ядерного столкновения энергию в этом диапазоне.

В области низких и высоких энергий распределение ошибки явно не может быть описано Гауссовым распределением. Наблюдается явная асимметрия и расширение хвостов распределений. Распределения смещены: для низких энергий в положительную сторону, а для высоких энергий в отрицательную.

Подобное поведение может быть связано с тем, что на краях энергетического спектра предсказания модели ограничены физическими границами: выделившаяся в ходе столкновения энергия не может быть меньше нуля (что ограничивает левый “хвост” в области низких энергий) и не может быть больше $\sqrt{s_{total}} = 2168\ GeV$ (что ограничивает правый “хвост” в области высоких энергий). Такие ограничения не учитываются моделью во время обучения, но при этом влияют на итоговый результат: на краях диапазона модель теряет способность аппроксимировать значения энергии, выделившейся в ходе столкновения, особенно когда значения численно близки к значениям на краях граничных условий.

Для более глубокого анализа формы распределения ошибки предсказания были предприняты попытки аппроксимации гистограмм с использованием не только Гауссова распределения, но и Лоренцева. Для количественной оценки качества аппроксимации использовался скорректированный коэффициент детерминации (Adjusted R^2).

Ниже приведены результаты для полносвязной НС, обучавшейся только на заряженных частицах. В остальных случаях результаты примерно те же.

Adjusted R^2	0.985	0.977	0.963
Lorentz			

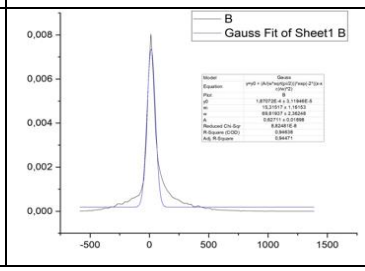
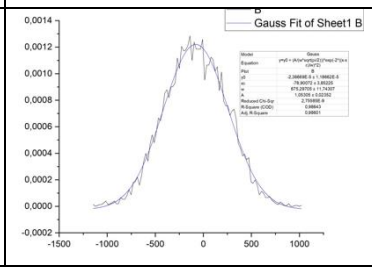
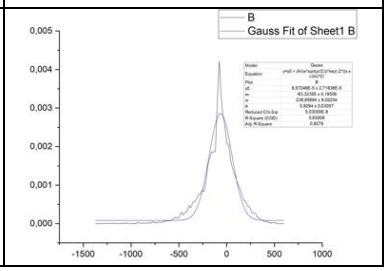
Adjusted R^2	0.945	0.986	0.928
Gauss			

Рис. 17: Гистограммы распределения ошибки предсказания с аппроксимацией Гауссовым и Лоренцевым распределениями для трех диапазонов энергий (слева направо: низкие, средние, высокие).

В области средних энергий, в согласии с предыдущим наблюдением, аппроксимация нормальным распределением дает хороший результат: $Adjusted\ R^2 = 0.986$. В области высоких энергий ни одна из функций не дает качественного приближения (Lorentz: $Adjusted\ R^2 = 0.963$, Gauss: $Adjusted\ R^2 = 0.928$), что указывает на существенное искажение формы распределения. Это может быть связано с физическим ограничением в этой области энергии, переобучением модели, недостатком обучающих примеров. В области низких энергий распределение ошибки хорошо аппроксимируется распределением Лоренца: $Adjusted\ R^2 = 0.985$. Это свидетельствует о наличие выраженных пиков и тяжелых хвостов. То, что в области низких энергий распределение ошибки лучше аппроксимируется функцией Лоренца может указывать на то, что в ходе обучения нейронной сети не учитывается физическое ограничение энергии.

Выводы и дальнейшие направления исследования

В ходе выполнения данной работы была получена теоретическая зависимость $\sqrt{s_{eff}}\ (b)$. Были проверены законы сохранения энергии и импульса для генератора QGSM, а также, учитывая оценку ЗС, были сформулированы критерии (ЗСЭ выполняется в пределах 1.5%, а ЗСИ - 14%), по которым необходимо определять энергию, выделившуюся в ходе столкновения. Однако, для ЗСИ опытным путем было выявлено, что графики зависимости энергии столкновения от прицельного параметра получаются лучше в пределах погрешности 2%. Это может быть связано с точность данных. Также были построены графики зависимости энергии, выделившейся в столкновении, от прицельного параметра столкновения: $\sqrt{s_{eff}}(b)$. Полученные зависимости совпадают с теоретическими.

Во второй части работы была построена функция, которая отображает сигнал, получаемый с детектора на микроканальных пластинках, на выделившуюся в столкновении энергию. Рассматривалось два вида модели: полносвязная и с добавлением сверточного слоя. Нейронная сеть обучалась сначала на всем наборе данных, а потом только на данных, содержащих исключительно заряженные частицы. После проведения ряда экспериментов было выявлено, что модель лучше всего работает, если обучается на всем наборе данных, с добавлением сверточного слоя в архитектуру. В этом случае относительная погрешность $\varepsilon = 6.95\%$, а среднеквадратичная ошибка $RMSE = 150.5$. Оба значения лежат в пределах допустимой нормы, а значит модель достаточно точно численно оценивает исследуемую величину.

Также был проведен глубокий анализ трех диапазонов энергий: низких, средних и высоких. Особый интерес представляют гистограммы распределения ошибки для всех трех областей. Для области средних энергий наблюдается симметричная форма распределения, а разброс значений является достаточно небольшим: распределение ошибок хорошо аппроксимируется гауссианом. Это указывает на устойчивость модели в области средних энергий и на ее способность достаточно хорошо предсказывать значения выделившейся энергии в этой области. Для низких энергий распределение ошибки лучше всего аппроксимируется функцией распределения Лоренца, а для высоких энергий – ни Лоренцом, ни Гауссом. Это можно объяснить физическими ограничениями, которые не учитываются моделью явно во время процесса обучения: выделившаяся в ходе столкновения двух ядер золота энергия не может быть меньше нуля и больше $\sqrt{s_{total}} = 2168$ GeV. Таким образом, в диапазоне малых энергий происходит утяжеление “хвостов” и смещение гистограммы вправо, а в области высоких энергий – влево.

Результаты, полученные в ходе данной работы, открывают возможности для дальнейшего усовершенствования модели. Например, можно попробовать заменить функцию активации SiLU на другую, обладающую лучшими свойствами в области высоких и низких энергий: это может положительно сказаться на способности модели интерпретировать входные данные и строить зависимости. Кроме того, важным шагом может стать предварительная нормализация данных. Приведение данных к диапазону значений $[0; 1]$ способствует более стабильному и эффективному процессу обучения нейронной сети. Также имеет смысл оперировать не энергией, приходящейся на два ядра золота, а на нуклон-нуклонную пару. Одним из вариантов является использование других архитектур, например с большим количеством сверточных слоев.

Еще одним важным направлением для дальнейших исследований является пересмотр целевого функционала. В настоящей работе применялась функция потерь MSE, предполагающая симметричное распределение ошибки. Однако, реальные распределения ошибки обладают асимметрией в областях, где проявляется физическое ограничение энергий. Таким образом, в дальнейшем планируется разработать подход, учитывающий асимметрию в данных.

Используемое оборудование

В ходе данной работы для достижения поставленных задач использовался язык программирования Python, а также следующие библиотеки:

- Numpy [6]
- Pandas [7]
- Matplotlib [8]
- PyTorch [9]
- Scipy [10]
- Scikit-learn [11]

Литература

[1] Садбери А. Квантовая механика и физика элементарных частиц. – 1989.

[2] Galaktionov K. A., Roudnev V. A., Valiev F. F. Neural Network Approach to Impact Parameter Estimation in High-Energy Collisions Using Microchannel Plate Detector Data //Moscow University Physics Bulletin. – 2023. – Т. 78. – №. Suppl 1. – С. S52-S58.

[3] Mallick N. et al. Estimating elliptic flow coefficient in heavy ion collisions using deep learning //Physical Review D. – 2022. – Т. 105. – №. 11. – С. 114022.

[4] Hirvonen H., Eskola K. J., Niemi H. Deep learning for flow observables in ultrarelativistic heavy-ion collisions //Physical Review C. – 2023. – Т. 108. – №. 3. – С. 034905.

[5] NICA[Электронный ресурс] – URL: https://nica.jinr.ru/ru/

[6] Numpy[Электронный ресурс] – URL: https://numpy.org/

[7] Pandas[Электронный ресурс] – URL: https://pandas.pydata.org/

[8] Matplotlib[Электронный ресурс] – URL: https://matplotlib.org/

[9] PyTorch[Электронный ресурс] – URL: https://pytorch.org/

[10] Scipy[Электронный ресурс] – URL: https://scipy.org/

[11] Scikit-learn[Электронный ресурс] – URL: https://scikit-learn.org/

Приложение 1

Рассмотрим вывод зависимости $\sqrt{s_{eff}}\,(b)$:

Для столкновения двух сферических частиц имеет место соотношение для площади пересечения:

$$Area \sim R^2 - b^2 = R^2\left(1 - \frac{b^2}{R^2}\right)$$

здесь $Area$ – площадь пересечения, R – радиус сферической частицы, b – прицельный параметр

Число частиц, участвующих во взаимодействии:

$$N_{part} \sim \rho_{nucleon} \cdot Volume;\ Area \sim Volume,$$

здесь N_{part} – число частиц, участвующих во взаимодействии, $\rho_{nucleon}$ – нуклонная плотность, $Volume$ – объем. Тогда получаем соотношение:

$$N_{part} \sim Area$$

Число бинарных нуклон-нуклон взаимодействий определяется через общее число взаимодействующих частиц следующим образом:

$$N_{coll} \sim N_{part}^{\alpha},$$

здесь $\alpha(b) \in [1;\ 2]$ – некоторая функция, для которой $\alpha|_{b=0} = 2$ и $\alpha|_{b=R} = 1$. Часто для описания числа взаимодействий внутри промежутка принимают $\alpha = 1.5$

Эффективная энергия зависит от числа бинарных нуклон-нуклонных столкновений и от энергии на нуклон-нуклонную пару:

$$\sqrt{s_{eff}} \sim N_{coll}\sqrt{s_{NN}},$$

Здесь $\sqrt{s_{eff}}$ – энергия в системе центра масс частиц, участвующих во взаимодействии.

Таким образом, объединяя все вышеупомянутые соотношения получаем:

$$\sqrt{s_{eff}} \sim N_{part}^{\alpha}\sqrt{s_{NN}} \sim (R^2 - b^2)^{\alpha}\sqrt{s_{NN}}$$

Приложение 2

```
import math
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import scipy as sp
import seaborn as sns
import rarfile
from scipy.stats import norm
import pandas as pd
Z = 79
A = 197
C = 29.9792 # (cm/ns)
COMP_EN = 11.006
Rel_Err = 0.05 #Relative error (instead of eps = 1e-8)
# define detector geometry

D1 = 5      #inner hole (cm)
D2 = 50     #outer hole (cm)
num_rings = 6  #number of radial rings
num_sectors = 10  #number of sectors(angular divisions)
R1 = D1/2       #inner radius (cm)
R2 = D2/2       #outer radius (cm)
z_detector1 = -400  #distance from the collision point (cm) - 1st detector
z_detector2 = 400   #distance from the collision point (cm) - 2st detector
z_detector = z_detector1 = z_detector2

# define ring and sector boundaries

ring_edges = np.linspace(R1, R2, num_rings + 1)
sector_edges = np.linspace(0, 2 * math.pi, num_sectors + 1)
print(ring_edges)
print(sector_edges)
# particle trajectory simulation


# a function that projects the trajectory of a particle
# and return (x,y) and time of flight (tof)

def project_to_detector(p_x, p_y, p_z, Mass, Z_detector):

    if p_z == 0:
        print("will never cross")       # particle will never touch the detector

    time_parameter = Z_detector / p_z
    x_hit = time_parameter * p_x
    y_hit = time_parameter * p_y
```

```
    C = 29.9792 # (cm/ns)
    p = np.sqrt(p_x**2 + p_y**2 + p_z**2)
    energy = np.sqrt(p**2 + Mass**2)
    velocity = (p / energy) * C      # particle velocity in (cm/ns)
    tof = abs(Z_detector) / velocity # time of flight in nanoseconds
    t0 = abs(Z_detector) / C # our shift in time -> the time it takes for the light
to reach the detector
    #print('Time = ',tof,'s; shift = ', t0, 's.')
    Tof = 1 / (tof - t0)
    return x_hit, y_hit, Tof
# particle trajectory simulation
# next step - define the pad ; (x,y) - its x_hit and y_hit
# try to determine which pad particle hits. pad = (ring; sector)

def find_pad(x, y):
    # conversion to polar coordinates
    r = np.sqrt(x**2 + y**2)
    fi = np.arctan2(y, x)  #returns the angle between [-pi; pi]

    # fi : [0; 2pi]
    if fi < 0:
        fi += 2 * np.pi

    ring_num = np.digitize([r], ring_edges) - 1
    sector_num = np.digitize([fi], sector_edges)
    # checking if the particle hit the detector
    if ring_num[0] <= 0 or ring_num[0] > num_rings:
        return None
    else:
        return [ring_num[0].item(), sector_num[0].item()]
def create_feature_vector(filtered_pads_tof, pz_values, max_particles = 120):

    # create a table filed with zeros
    feature_matrix = np.zeros((3, max_particles))
    #print(feature_matrix)
    # pads and tof extraction
    pads = [pad for pad, _ in filtered_pads_tof] # pads extraction
    tof_s = [tof for _, tof in filtered_pads_tof] # tof extraction
    # number of particles restriction
    num_particles = min(len(tof_s), max_particles)
    #print(num_particles)
    for num in range(num_particles):

        # tof, ring, sector, pz extraction
        tof = tof_s[num]
        #ring, sector = pads[num]
        pz = pz_values[num]

        # detector def
        if pz > 0:
            detector_id = 1
```

```
        else:
            detector_id = -1

        # encode pad like 1 number

        pad_encoded = int(''.join(map(str, pads[num])))
        #print(pad_encoded)

            # make a vector
        feature_matrix[0][num] = tof
        feature_matrix[1][num] = pad_encoded
        feature_matrix[2][num] = detector_id
    feature_vector = feature_matrix.T.flatten()

    return feature_vector
def filter_min_tof(pads, tof_s):

        # create a dictionary to store the min tof for each pad: key -> pad; value
-> tof
    pad_to_tof = {}
        # iterate throught all pads and their tof values
    zipped = zip(pads, tof_s)
        #print(zipped)
    for i_pad, j_tof in zipped:

            pad_tuple = tuple(i_pad)    # convert list -> tuple, because dict use
cant use lists in keys
            # if pad is already in the dict => update if new tof is smaller
            if pad_tuple in pad_to_tof:
                pad_to_tof[pad_tuple] = min(pad_to_tof[pad_tuple], j_tof)
            else:
                pad_to_tof[pad_tuple] = j_tof # else add new pad to dict with its
tof


        # convert the dict into a list of lists
    filtered_pads_tof = [[list(pad), tof] for pad, tof in pad_to_tof.items()]


    return filtered_pads_tof
all_features = [] # final vector with all features (independent)
all_energies = [] # final vector with energy per collision (dependent)
files = ['f1.txt', 'f2.txt', 'f3.txt', 'f4.txt', 'f5.txt', 'f6.txt', 'f7.txt',
'f8.txt', 'f9.txt', 'f10.txt']

# Путь к архиву
rar_path = "C:\\VKR\\pythonProject1\\sourceQGSM_AuAu_11GeV_200k.rar"

# Открываем архив
```

```
for i in range(1, 101):
    file_index = f"D:\\200k\\data_{i:d}.dat"

    with open(file_index, 'r') as f:
        lines1 = f.readlines()

    i = 5
    #b_need = []
    pz_values = []
    #energies = []
    #all_events_pads = []
    #all_events_tof_s = []
    #pz_all = []
    while i < len(lines1):
        l = lines1[i]
        n = [float(num) for num in l.split()]
        n_part = int(n[1])
        #b = n[2]
        #bx = n[3]
        #by = n[4]
        #print(f' n = {n_part}')
        i += 1
        prev_i = i
        tot_en = 0  # полная энергия всех нуклонов
        all_pads = []
        tof_s = []
        pads = []

        while i < prev_i + n_part:
            l = lines1[i]
            params = [float(num) for num in l.split()]
            #type = int(params[4])
            mass = params[10]
            charge = params[0]
            px = abs(params[5])
            py = abs(params[6])
            pz = abs(params[7])
            condition1 = math.isclose(px, pz, abs_tol=0.9 * pz)
            condition2 = math.isclose(py, pz, abs_tol=0.9 * pz)
            #(type == 2212 or type == 2112)
            if (mass >= 9.38200E-01) and (not condition1) and (not condition2):
                energy_csm = (params[5]**2 + params[6]**2 + params[7]**2 +
mass**2)**(1/2)

                #print(energy_csm, '  ', COMP_EN)
                if (math.isclose(energy_csm, COMP_EN, rel_tol = COMP_EN *
Rel_Err)):
                    #print(b, energy_csm)
                    t = 0
                    #print(l)
                else:
```

```
                    tot_en += energy_csm
                    # pz_values.append(pz)
                    x, y, t = project_to_detector(px, py, pz, mass, z_detector)
                    #print(t)
                    pad = find_pad(x, y)
                    all_pads.append(pad)
                    # deleting [-100;100] - if particles didnt hit the detector
                    pads = [pad for pad in all_pads if pad is not None]
                    pz_values.append(pz)
                    tof_s.append(t.item())
                    #print(f'charge: {charge} and i = {i}')

            i += 1

        all_energies.append(tot_en)
        filtered = filter_min_tof(pads, tof_s)

        feature_vec = create_feature_vector(filtered, pz_values)
        all_features.append(feature_vec)
        #all_events_pads.append(pads)
        #all_events_tof_s.append(tof_s)
        #pz_all.append(pz_values)
with open('all_features.txt', 'w') as f:
    for i in range(len(all_features)):
        f.write(str(all_features[i]))
feature_matrixx = np.ones((len(all_features), 360))

for i in range(len(all_features)):
    feature_matrixx[i][:] = all_features[i]

print(feature_matrixx.shape)
# makes train and test sets

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(feature_matrixx, all_energies,
test_size=0.2, random_state=42)
# step 2 - create our own DataSet class
# features - list of feature vectors, one for each event <=> dim = [N, 3*120] N -
number of events
# energies - list of energies in every event <=> dim = N

class QGSMCollisionDataset(Dataset):
    def __init__(self, all_features, all_energies):
        self.X = torch.tensor(all_features, dtype = torch.float32)
        self.y = torch.tensor(all_energies, dtype = torch.float32).unsqueeze(1) #
[N, 1]
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
# step 3 - neural network architecture
```

```python
class EnergyRegressor(nn.Module):
    def __init__(self):
        super(EnergyRegressor, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(360, 128),
            nn.SiLU(),
            nn.Linear(128, 64),
            nn.SiLU(),
            nn.Linear(64, 32),
            nn.SiLU(),
            nn.Linear(32, 16),
            nn.SiLU(),
            nn.Linear(16, 1)
        )


    def forward(self, x):
        f_out = self.net(x)
        return f_out
class EnergyRegressor(nn.Module):
    def __init__(self):
        super(EnergyRegressor, self).__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=8, kernel_size=(6, 3)),
            nn.ReLU()
        )


        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(8 * 115 * 1, 128),
            nn.SiLU(),
            nn.Linear(128, 64),
            nn.SiLU(),
            nn.Linear(64, 32),
            nn.SiLU(),
            nn.Linear(32, 16),
            nn.SiLU(),
            nn.Linear(16, 1)
        )

    def forward(self, x):
        x = x.view(-1, 1, 120, 3)
        x = self.conv(x)
        x = self.fc(x)
        return x
# step 4 - training our model
```

```
def train_model(X_train, y_train, X_test, y_test, epochs = 1500, batch_size = 3000,
learning_rate = 0.001, patience = 100):

    dataset = QGSMCollisionDataset(X_train, y_train)
    loader = DataLoader(dataset, batch_size = batch_size, shuffle = True) # split
on batches and mix them
    X_test = torch.tensor(X_test, dtype=torch.float32)
    y_test = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

    model = EnergyRegressor()
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr = learning_rate, weight_decay=1e-
5)

    train_losses = []
    test_losses = []

    best_val_loss = float('inf')    # least validation error
    best_model_state = None # best weight models
    patience_counter = 0    # add one if the error does not decrease

    print("Minimum:", torch.min(torch.tensor(y_train, dtype=torch.float32)).item())
    print("Maximum:", torch.max(torch.tensor(y_train, dtype=torch.float32)).item())
    print("Mean:", torch.mean(torch.tensor(y_train, dtype=torch.float32)).item())

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for batch_X, batch_y in loader:
            #print('X.shape = ',batch_X.shape, "Y.shape = ", batch_y.shape)
            optimizer.zero_grad()       # reset gradients -> 0
            outputs = model(batch_X)    # make a prediction
            loss = criterion(outputs, batch_y)  # calculate out loss
            loss.backward()     # calculate gradients
            optimizer.step()    # update weights
            running_loss += loss.item()

        avg_train_loss = running_loss / len(loader) # average loss for 1 epoch
        train_losses.append(avg_train_loss) # for visualization

        # error estimation on the test sample
        model.eval()
        with torch.no_grad():
            test_preds = model(X_test)
            test_loss = criterion(test_preds, y_test).item()
        test_losses.append(test_loss)

        # early stopping
        if test_loss < best_val_loss:
            best_val_loss = test_loss
            best_model_state = model.state_dict()
```

```
            patience_counter = 0
        else:
            patience_counter += 1
            if patience_counter >= patience:
                print(f"early stopping triggered at epoch {epoch}")
                break

        if epoch % 100 == 0:
            print(f"running loss: {running_loss}", f"epoch: {epoch}")

    model.load_state_dict(best_model_state)

    # visualization
    plt.figure(figsize=(10, 5))
    plt.loglog(train_losses, color = 'red', label = 'Loss(TRAIN)')
    plt.loglog(test_losses, color = 'blue', label = 'Loss(TEST)')
    plt.title('Train VS Test Loss Evaluation')
    plt.xlabel('epoch')
    plt.ylabel('MSE - our loss')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()


    return model
r = train_model(X_train, y_train, X_test, y_test)
print(r)
# predicted and true energy values
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

r.eval()
with torch.no_grad():
    y_true = y_test.squeeze().numpy()
    y_pred = r(X_test).squeeze().detach().numpy()
min_energy = y_true.min()
max_energy = y_true.max()
step = (max_energy - min_energy) / 3
d1 = min_energy + step
d2 = d1 + step
d3 = d2 + step
print(f'Step: {step}')
print(f'Min Energy: {min_energy}')
print(f'Max Energy: {max_energy}')
print(f'd1 = {d1}')
print(f'd2 = {d2}')
print(f'd3 = {d3}')
from sklearn.metrics import mean_squared_error
# metric intro on the test set
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
```

```
print(f"RMSE: {rmse:.5f} GeV")
def plot_error_hist(errors, label):
    #errors_np = errors.numpy()
    mu = np.mean(errors)
    sigma = np.std(errors)
    x = np.linspace(min(errors), max(errors), 1000)
    gaussian_curve = norm.pdf(x, mu, sigma)
    plt.figure(figsize=(8, 5))
    plt.hist(errors, bins=100, density=True, color='lightblue', edgecolor='black',
alpha=0.7, label='Errors')


    plt.plot(x, gaussian_curve, color='darkblue', linewidth=2.5, label='Gaussian
approximation')
    plt.axvline(0, color='red', linestyle='--', label='Zero error')
    plt.title(f"Error histogram with Gaussian approximation — {label} energy",
fontsize=14)
    plt.xlabel("prediction error (GeV)", fontsize=12)
    plt.ylabel("probability density", fontsize=12)
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
    #import pandas as pd

    #counts, bin_edges = np.histogram(errors, bins=100, density=True)
    #hist_data = pd.DataFrame({
    #    'Bin_Center': (bin_edges[:-1] + bin_edges[1:]) / 2,  # Центры интервалов
    #    'Counts': counts
    #})


    hist_data.to_csv('histogram_data.csv', index=False)

    #print("bin edges x:", bin_edges)
    #print("counts y:", counts)
# separation of energies by ranges (low, medium, high)

low_mask = y_true < d1
mid_mask = (y_true >= d1) & (y_true <= d2)
high_mask = y_true > d2

energy_ranges = {
    'Low Energy': low_mask,
    'Mid Energy': mid_mask,
    'High Energy': high_mask
}

for title, mask in energy_ranges.items():
    y_t = y_true[mask]
    y_p = y_pred[mask]
```

```
    errors = y_p - y_t

    plt.figure(figsize=(12, 5))

    # Scatter Plot
    plt.subplot(1, 2, 1)
    plt.scatter(y_t, y_p, s=2, alpha=0.3)
    plt.plot([y_t.min(), y_t.max()], [y_t.min(), y_t.max()], color='orange',
label='Perfect prediction')
    plt.xlabel("True Energy, GeV")
    plt.ylabel("Predicted Energy, GeV")
    plt.title(f"{title} - Scatter")
    plt.legend()

    # Density + Contour
    plt.subplot(1, 2, 2)
    sns.kdeplot(x=y_t, y=y_p, fill=True, cmap="Purples", levels=20, thresh=0.01,
linewidths=2.5, linestyles='solid')
    plt.plot([y_t.min(), y_t.max()], [y_t.min(), y_t.max()], color='orange',
label='Perfect prediction')
    plt.xlabel("True Energy")
    plt.ylabel("Predicted Energy")
    plt.title("Density Plot")
    plt.legend()

    # Histogram

    plot_error_hist(errors, title)


plt.tight_layout()
plt.show
```